

Introduction to audio processing with neural networks

Ulysse Rançon, Ismail Khalfaoui Hassani, Thomas Pellegrini

The first speech recognition system



1961: IBM shoebox

- recognizes 16 words
- executes simple mathematical operations
- heavily influenced the field of speech processing



<https://www.youtube.com/watch?v=rQco1sa9AwU>

The first speech recognition system



1961: IBM shoebox

- recognizes 16 words
- executes simple mathematical operations
- heavily influenced the field of speech processing



<https://www.youtube.com/watch?v=rQco1sa9AwU>

2025: vocal assistants



Outline

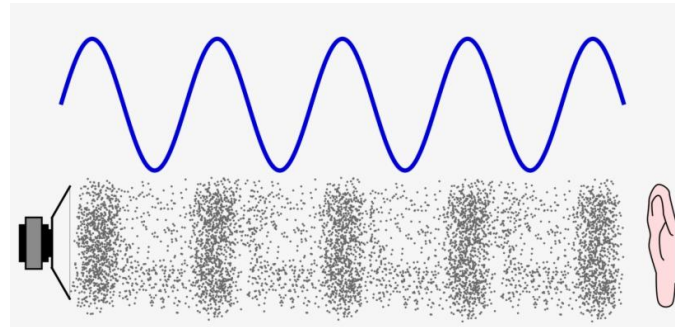
- I. What is sound ?
- II. Reminders on signal acquisition
- III. Auditory representations
- IV. Encoding audio with neural networks
- V. Hands-on practice

I. what is sound ?

some physics reminders

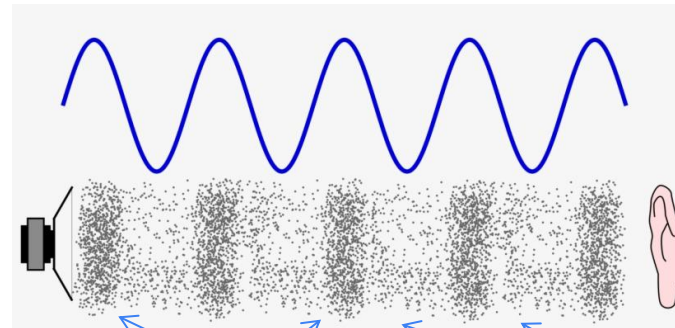
What is sound?

a **mechanical** (air pressure) wave, travelling at **XXX** m/s



What is sound?

a **mechanical** (air pressure) wave, travelling at 340 m/s



high pressure
Audio processing

low pressure
Ulysse Rançon

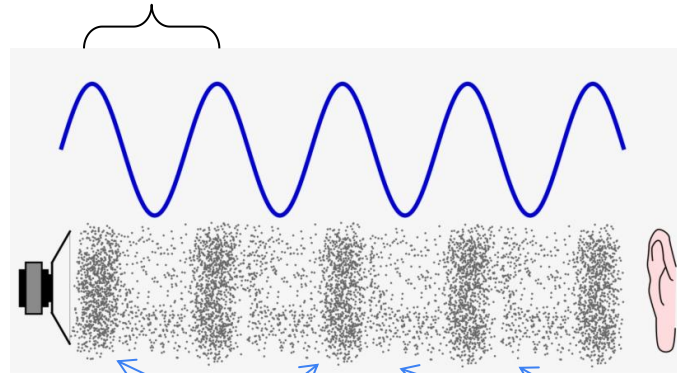
What is sound?

a **mechanical** (air pressure) wave, travelling at 340 m/s



?

wave length = f (frequency)



high pressure

Audio processing

low pressure

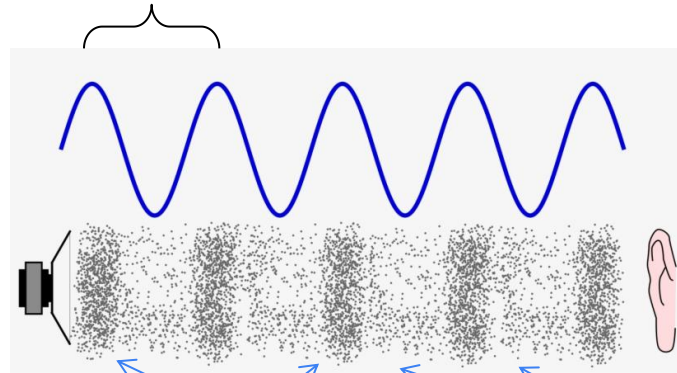
Ulysse Rançon

What is sound?

a **mechanical** (air pressure) wave, travelling at 340 m/s



wave length = $\text{speed} / \text{frequency}$



high pressure
Audio processing

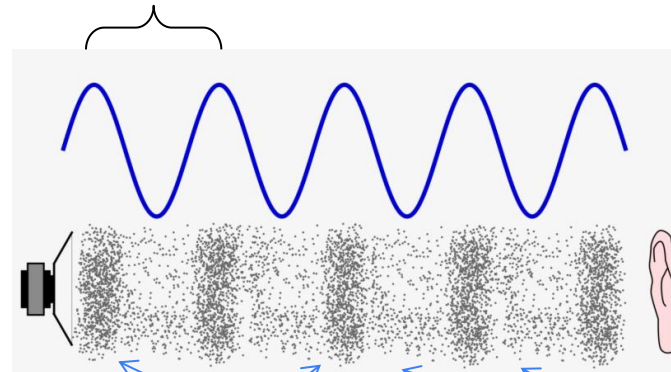
low pressure
Ulysse Rançon

What is sound?

a **mechanical** (air pressure) wave, travelling at 340 m/s



$$\text{wave length} = \text{speed} / \text{frequency}$$



Characteristics

Waves

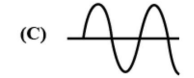
1) High pitch low volume



2) Low pitch high volume



3) Low pitch low volume



4) High pitch high volume



small exercise:

link each description to a wave

II. Signal acquisition

basics of digital signal processing

Reminders of signal acquisition

Step 1: Analog audio signal - Continuous representation of signal



Step 2: Sampling - Samples are selected at regular time intervals



Step 3: Digital audio signal - The way it is stored in memory



Analog **vs** digital

Reminders of signal acquisition

Step 1: Analog audio signal - Continuous representation of signal



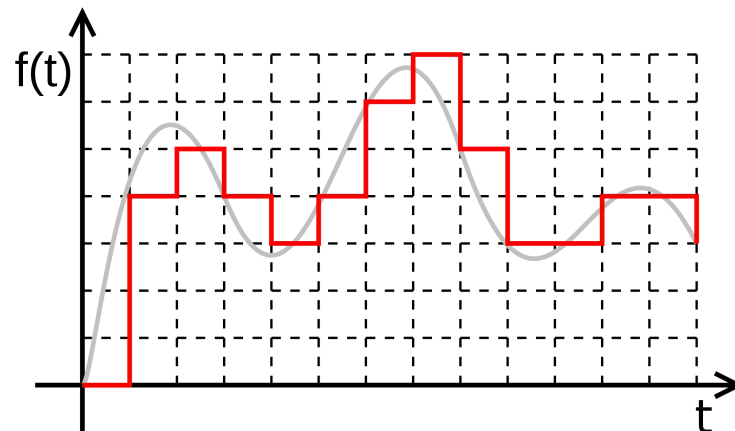
Step 2: Sampling - Samples are selected at regular time intervals



Step 3: Digital audio signal - The way it is stored in memory



Analog vs digital



real value, continuous over time, the actual signal itself

Discrete time intervals and quantized values (coded in bits)

Reminders of signal acquisition

Step 1: Analog audio signal - Continuous representation of signal



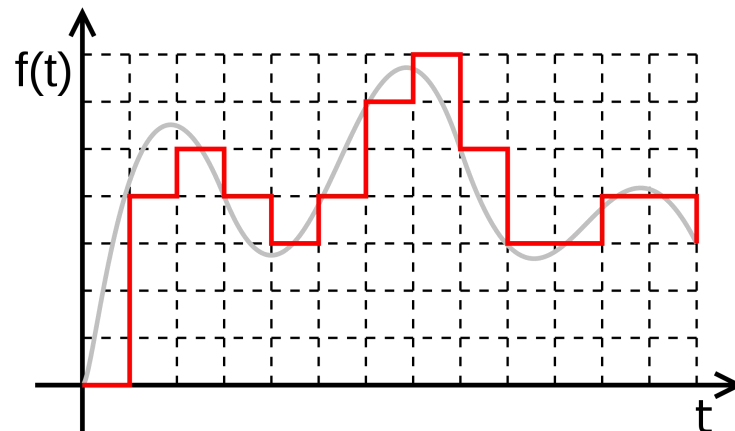
Step 2: Sampling - Samples are selected at regular time intervals



Step 3: Digital audio signal - The way it is stored in memory



Analog vs digital



quantization error?

real value, continuous over time, the actual signal itself

Discrete time intervals and quantized values (coded in bits)

Reminders of signal acquisition

Step 1: Analog audio signal - Continuous representation of signal



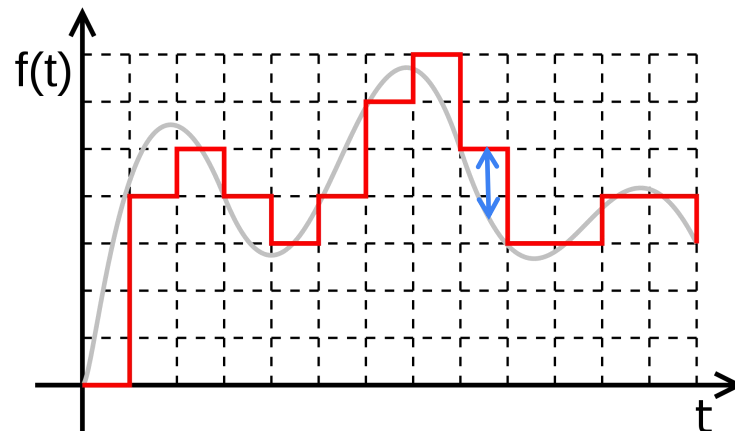
Step 2: Sampling - Samples are selected at regular time intervals



Step 3: Digital audio signal - The way it is stored in memory



Analog vs digital



quantization error?

real value, continuous over time, the actual signal itself

Discrete time intervals and quantized values (coded in bits)

Reminders of signal acquisition

Step 1: Analog audio signal - Continuous representation of signal



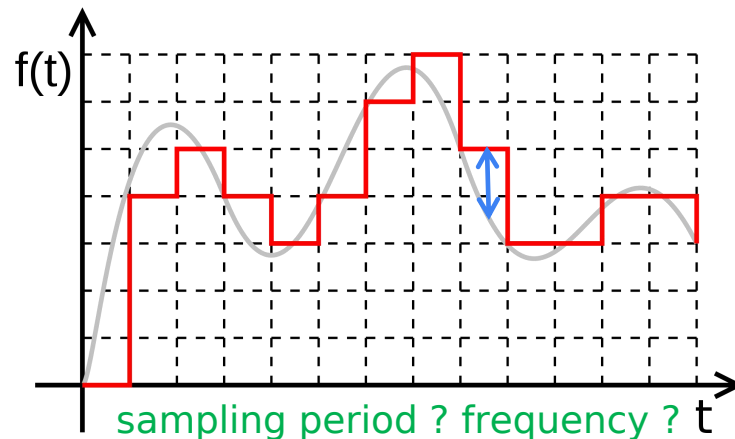
Step 2: Sampling - Samples are selected at regular time intervals



Step 3: Digital audio signal - The way it is stored in memory



Analog vs digital



quantization error ?

real value, continuous over time, the actual signal itself

Discrete time intervals and quantized values (coded in bits)

Reminders of signal acquisition

Step 1: Analog audio signal - Continuous representation of signal



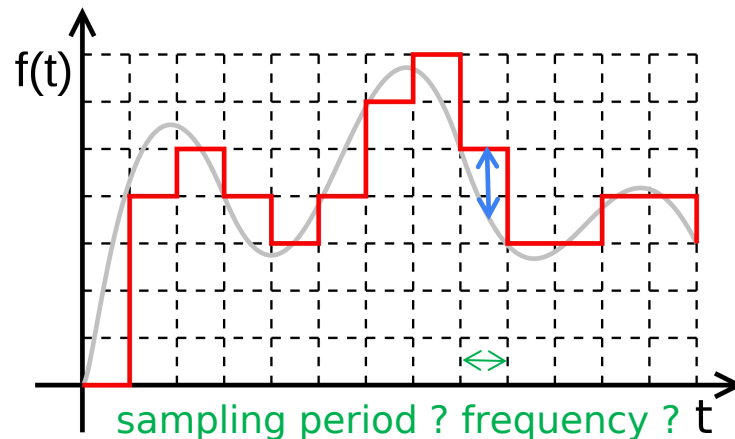
Step 2: Sampling - Samples are selected at regular time intervals



Step 3: Digital audio signal - The way it is stored in memory



Analog vs digital



quantization error ?

real value, continuous over time, the actual signal itself

Discrete time intervals and quantized values (coded in bits)

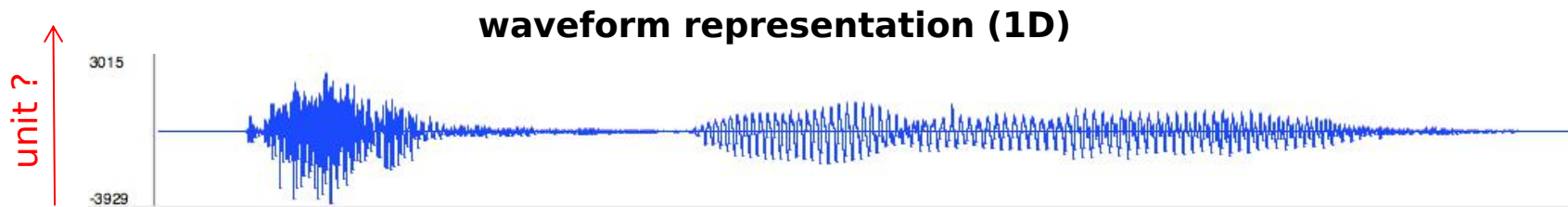
III. Auditory representations

the building blocks of audio processing

How to encode audio?

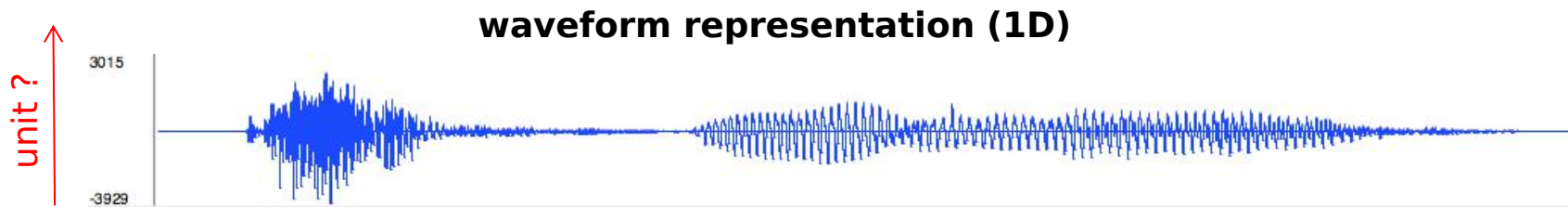
2 main families of **representations** for audio signals

1) The “raw” waveform



represents xxx over time

1) The “raw” waveform

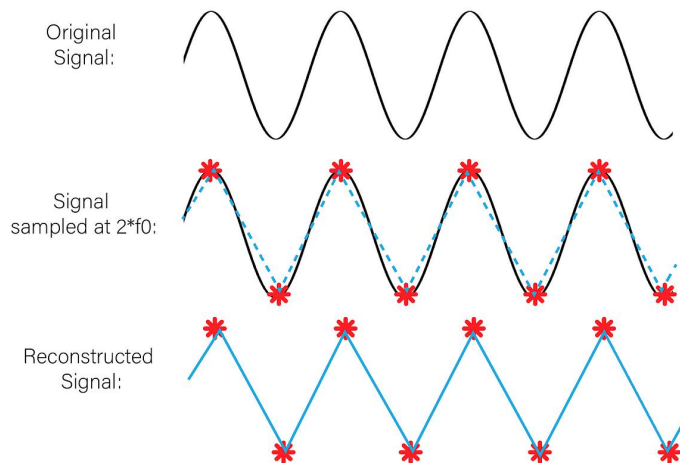


*represents **sound wave amplitude** over time*

The appropriate sampling frequency for audio

Shannon-Nyquist Theorem:

At a sampling rate of sr [Hz], the maximum frequency that can be 'captured' = $sr/2$ [Hz]

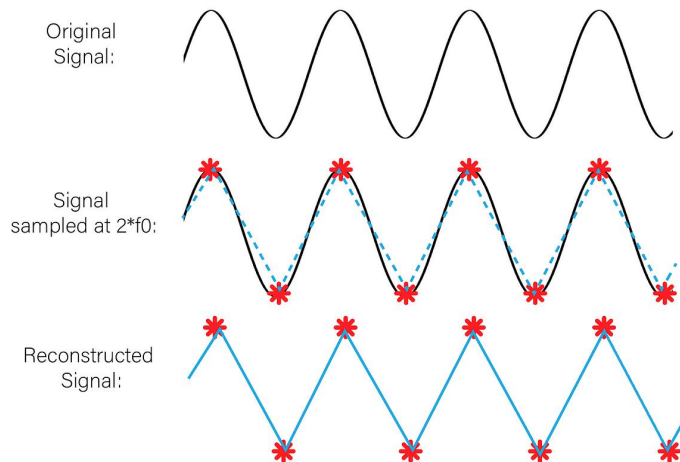


The appropriate sampling frequency for audio

Shannon-Nyquist Theorem:

At a sampling rate of sr [Hz], the maximum frequency that can be 'captured' = $sr/2$ [Hz]

Human hearing range ?



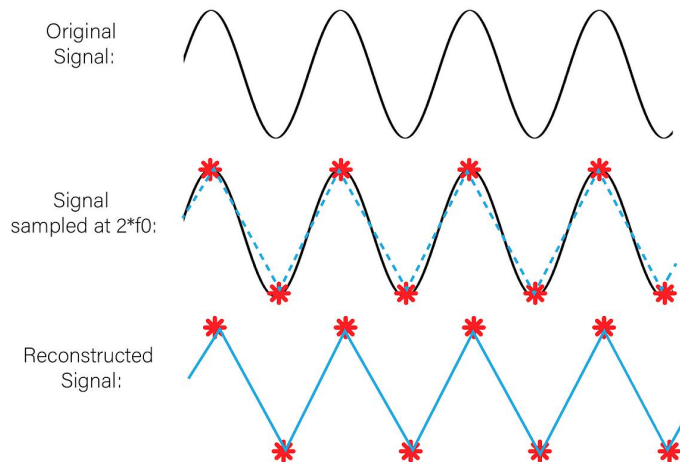
The appropriate sampling frequency for audio

Shannon-Nyquist Theorem:

At a sampling rate of sr [Hz], the maximum frequency that can be 'captured' = $sr/2$ [Hz]

Human hearing range ?

--> [20 Hz, 20 kHz]



The appropriate sampling frequency for audio

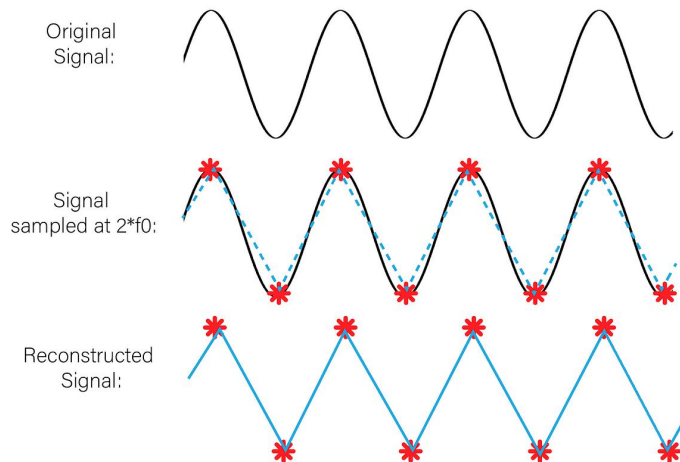
Shannon-Nyquist Theorem:

At a sampling rate of sr [Hz], the maximum frequency that can be 'captured' = $sr/2$ [Hz]

Human hearing range ?

--> [20 Hz, 20 kHz]

Minimum sampling frequency necessary to capture it ?



The appropriate sampling frequency for audio

Shannon-Nyquist Theorem:

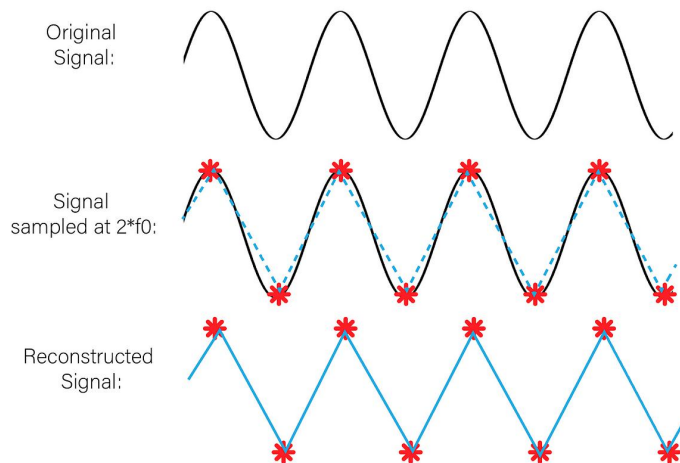
At a sampling rate of sr [Hz], the maximum frequency that can be 'captured' = $sr/2$ [Hz]

Human hearing range ?

--> [20 Hz, 20 kHz]

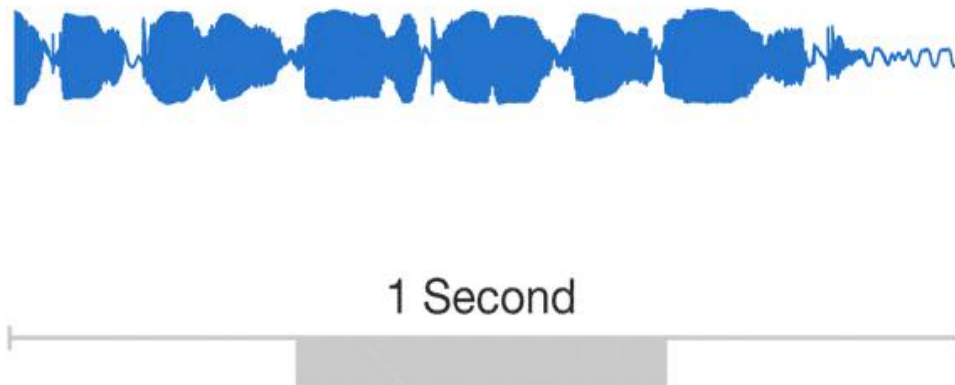
Minimum sampling frequency necessary to capture it ?

--> 44.1 kHz



Typical sample rates

Raw audio signals are usually between 16K to 44K points per second!



Source: <https://deepmind.com/blog/article/wavenet-generative-model-raw-audio>

On storing audio data

Raw audio is surprisingly very heavy !!

Exercise:

how much memory takes an uncompressed raw audio song

- of 5 mins
- sampled @ 44.1 kHz
- and having each sample quantized on 32 bits ?

--> provide answer in Mo

On storing audio data

- Raw audio is surprisingly very heavy !!

Exercise:

how much memory takes an uncompressed raw audio song

- of 5 mins
- sampled @ 44.1 kHz
- and having each sample quantized on 32 bits ?

--> provide answer in Mo

$$5 \times 60 \times 44100 \times 32 / 8000 = 52.9 \text{ Mo !!}$$

--> How come my mp3 files are so lightweight on my phone ???

On storing audio data

--> How come my mp3 files are so lightweight on my phone ???

It's a matter of data **compression**

wav files are uncompressed data format. Very high quality, but also very heavy.

Compressed formats like **mp3** find tricks to code the same data with less memory

- redundant patterns
- discard useless information

On storing audio data

--> How come my mp3 files are so lightweight on my phone ???

It's a matter of data **compression**

wav files are uncompressed data format. Very high quality, but also very heavy.

Compressed formats like **mp3** find tricks to code the same data with less memory

- redundant patterns
- discard useless information



remove frequencies in the barely audible range !
(human hearing range: 20 - 20,000 Hz)

On storing audio data

--> How come my mp3 files are so lightweight on my phone ???

It's a matter of data **compression**

wav files are uncompressed data format. Very high quality, but also very heavy.

Compressed formats like **mp3** find tricks to code the same data with less memory

- redundant patterns
- discard useless information



remove frequencies in the barely audible range !
(human hearing range: 20 - 20,000 Hz)

Usual sampling
freq. is 44.1 kHz



Some inspiration from the brain

- Biological systems (our brains and ears) excel at processing sounds
--> take some inspiration

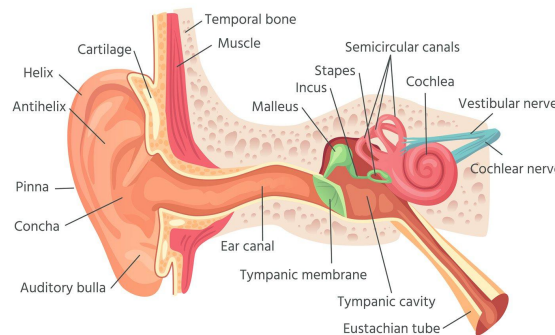
Some inspiration from the brain

- Biological systems (our brains and ears) excel at processing sounds
--> take some inspiration
- Notably, our brain does not “work” on the raw waveform !
--> It splits it into multiple frequencies

Some inspiration from the brain

- Biological systems (our brains and ears) excel at processing sounds
--> take some inspiration
- But our brain does not “work” on the raw waveform !
--> It splits it into multiple frequencies

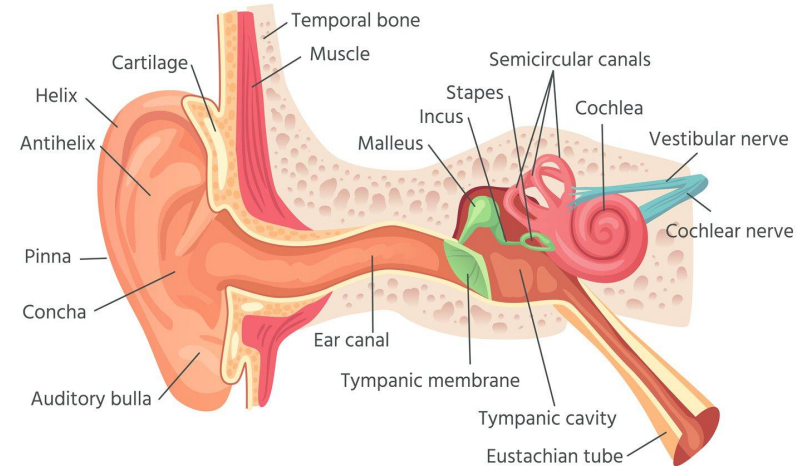
That's what the (inner) ear, or **cochlea**, does !!



The peripheral auditory system

Exercise: describe the traveling of sound in the **peripheral auditory system**

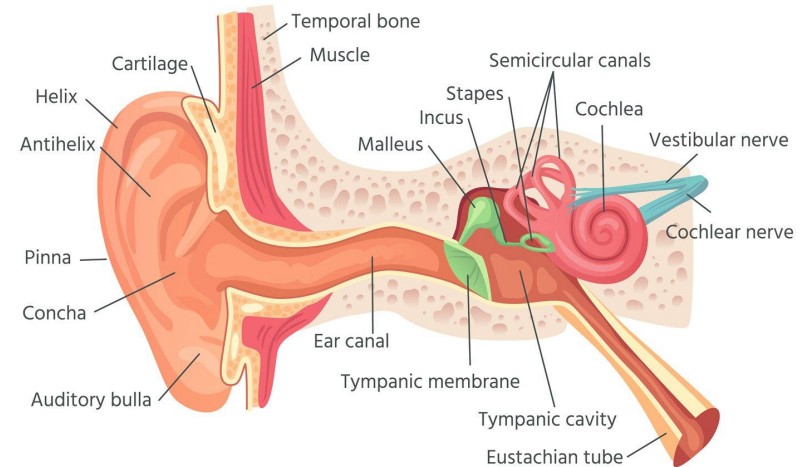
1. -
2. -
3. -
4. -



The peripheral auditory system

Quick reminders:

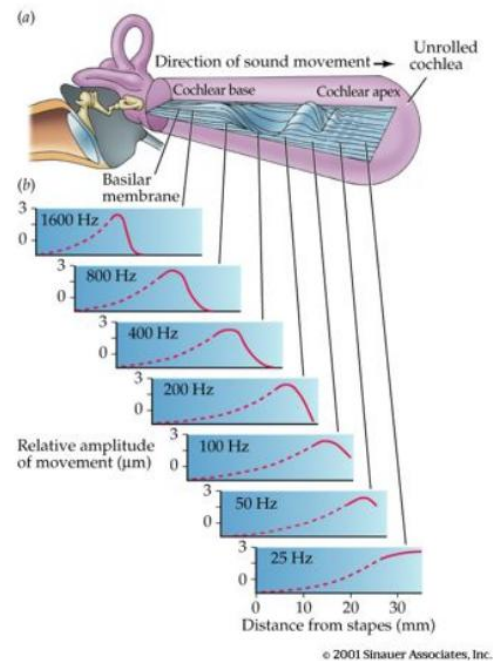
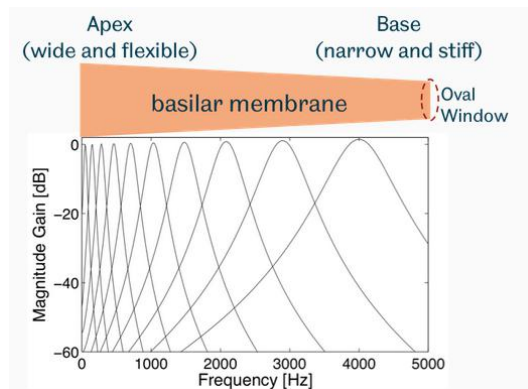
1. Sound travels down the **auditory canal**
2. causes the **tympanic membrane** to vibrate
3. moving the **ossicles**
4. creating a pressure wave travelling on the **basilar membrane** of the **cochlea**.



The peripheral auditory system

The cochlea is a fluid chamber organ; its snail shell shape allows to decompose sound into different frequencies (wavelet transform)
→ a **natural spectral analyzer**

Each position along the cochlea is sensitive for a specific frequency



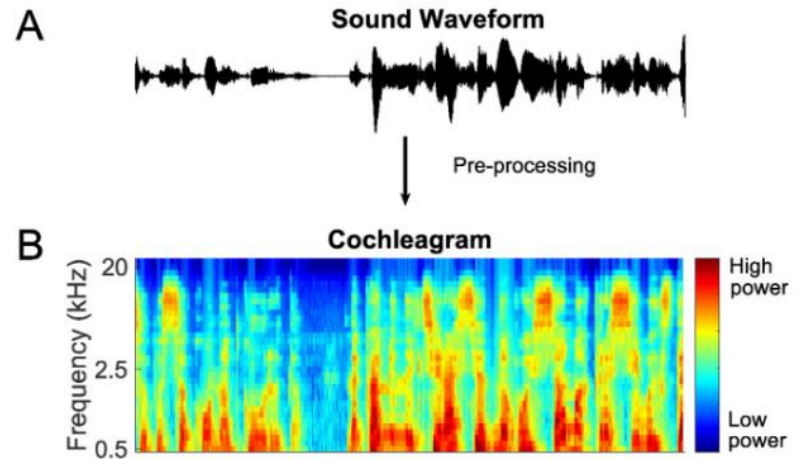
© 2001 Sinauer Associates, Inc.

A motivation for spectrograms



Georg von Békésy,
the “discoverer” of the cochlea
Medicine Nobel Prize (1961)

The animal ear is highly optimized, so let's mimick it !

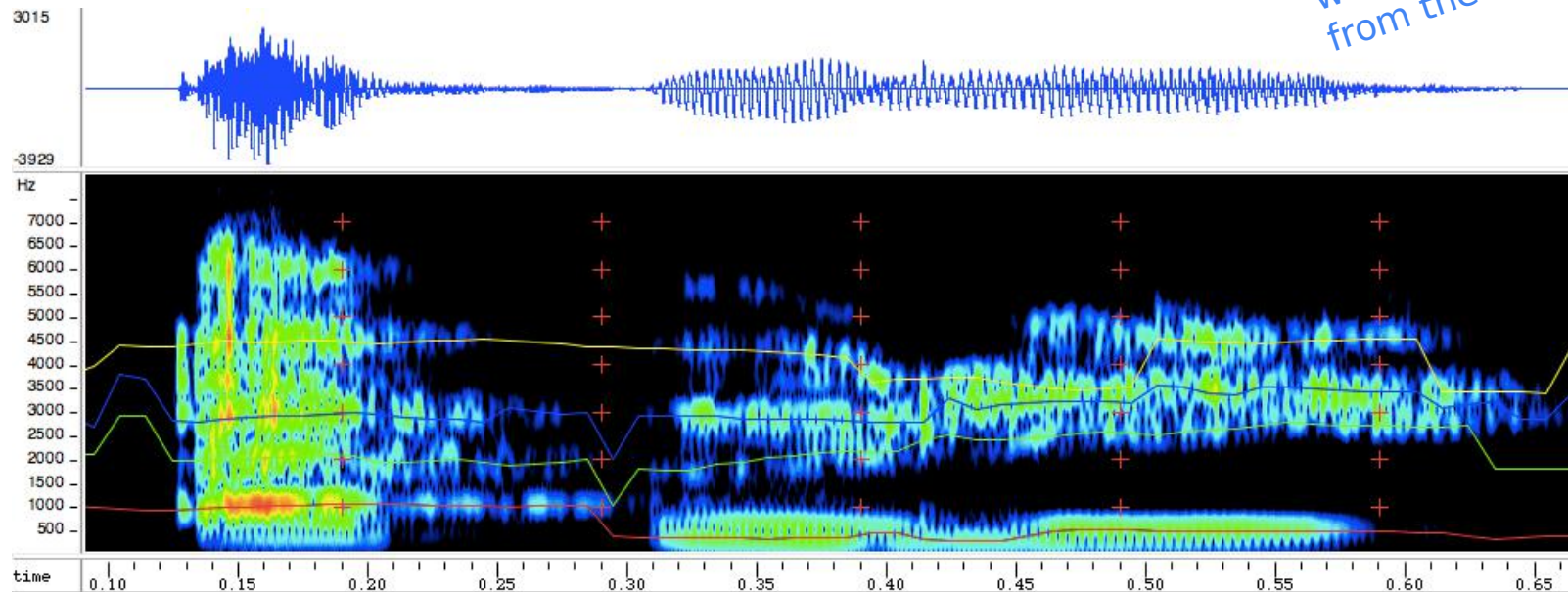


to go further:

Artem Kirsanov's YouTube channel about neuroscience
<https://www.youtube.com/watch?v=jnxqHcObNK4>

2) spectrograms

waveform representation (1D)

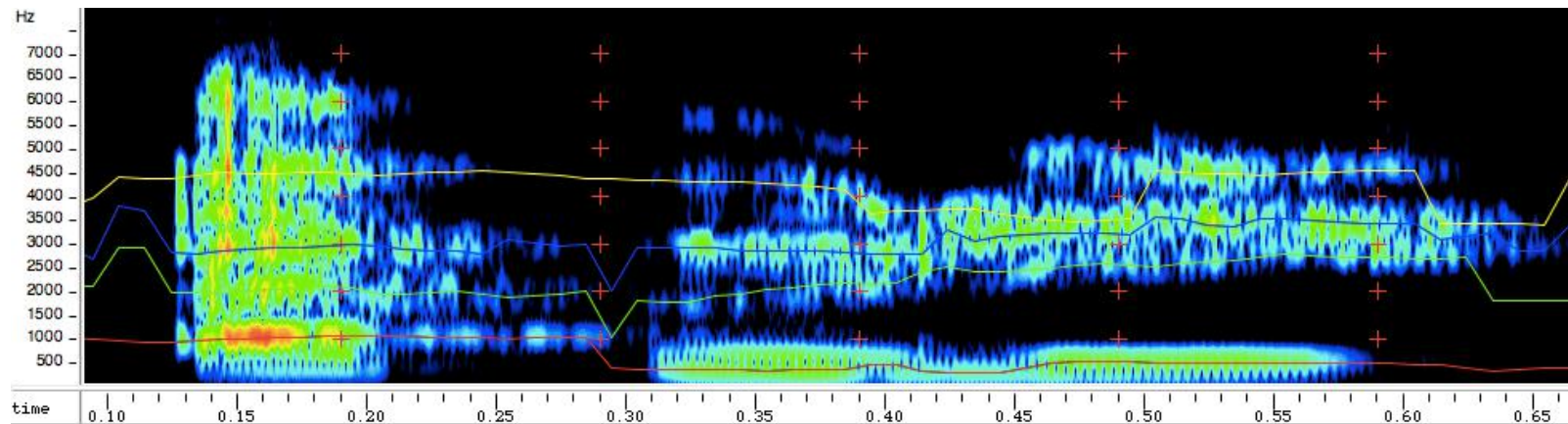


*we always start
from the waveform !*

*Fourier,
wavelets,
...*

spectrogram representation (2D)

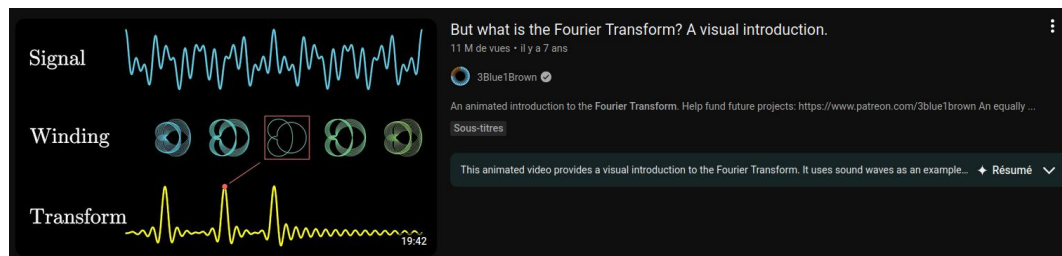
2) spectrograms



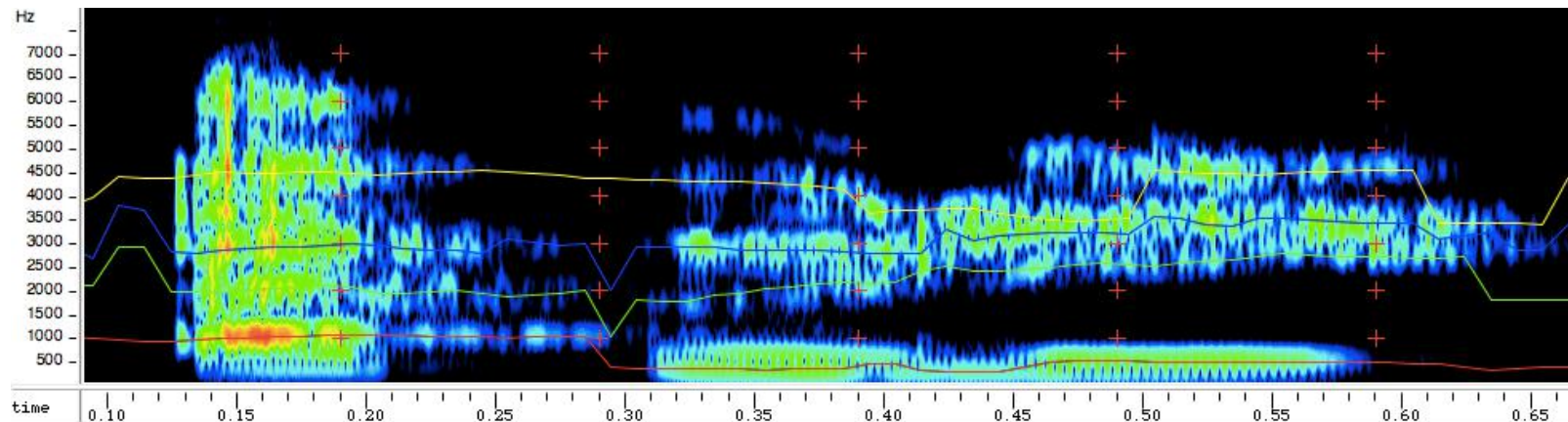
spectrogram representation (2D)

2) spectrograms

represents the (approximate) frequency content of the signal over time



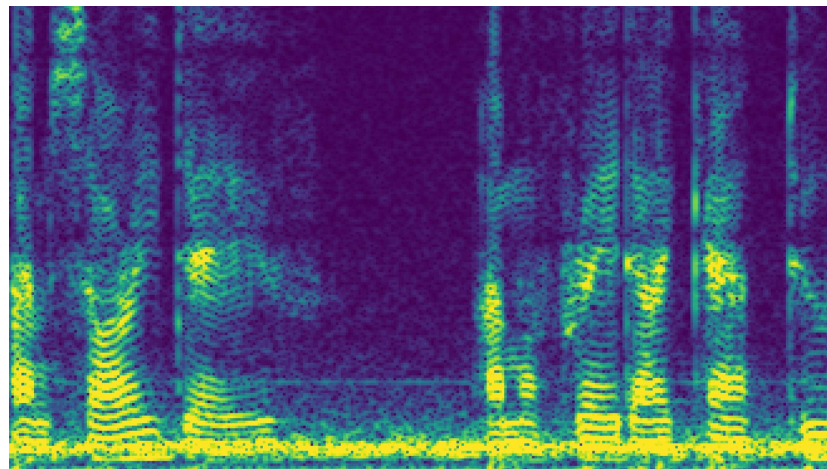
https://youtu.be/spUNpyF58BY?si=-XbwL9s70XqL6-w_



spectrogram representation (2D)

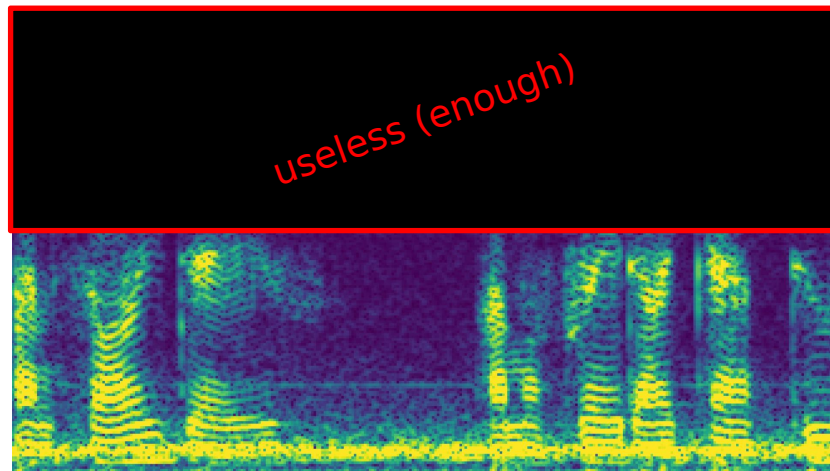
Back to data compression

the frequency range of human voice is [2,000 - 6,000] Hz
--> for a song or a phone call, discard other frequencies



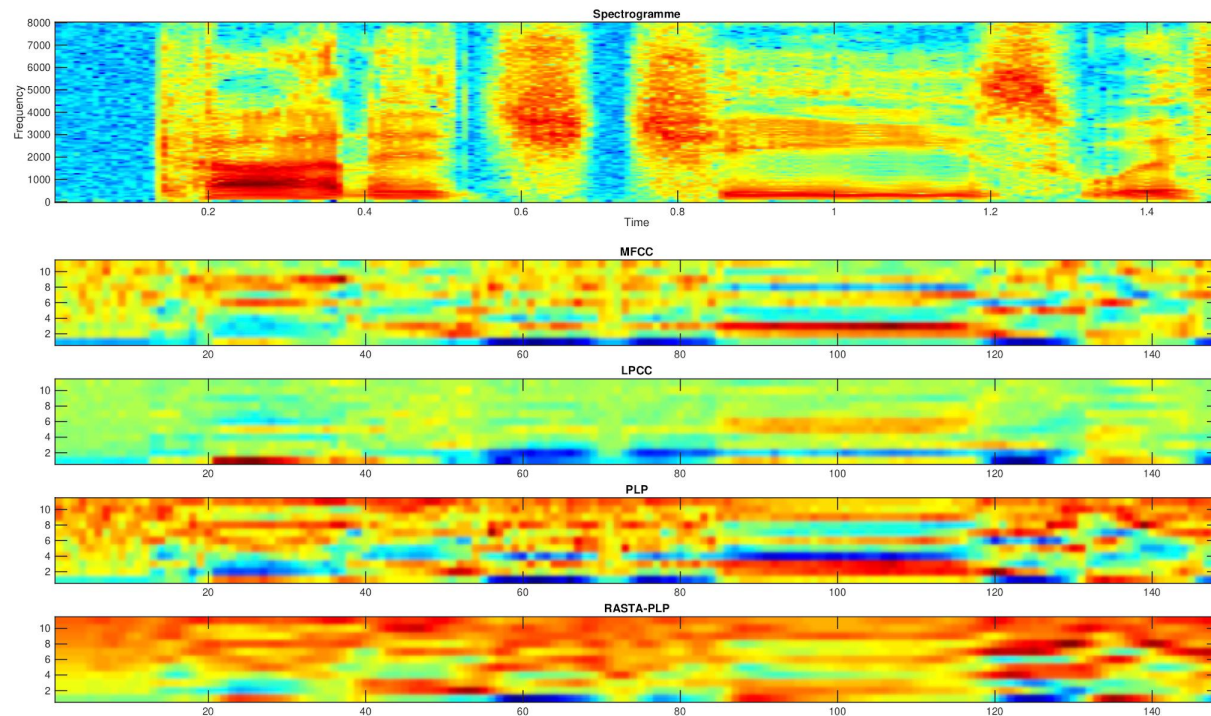
Back to data compression

the frequency range of human voice is [2,000 - 6,000] Hz
--> for a song or a phone call, discard other frequencies



Many possible “spectrogram-like” representations !

Various methods allow to go from the waveform to a spectrogram



- Short-Term Fourier Transform (STFT),
- Gammatone spectrogram,
- Mel spectrogram,
- Mel Cepstral coefficients (MFCCs)...

$$\text{spectro} = f(\text{wav})$$

*can be very complex !
(lots of operations)*

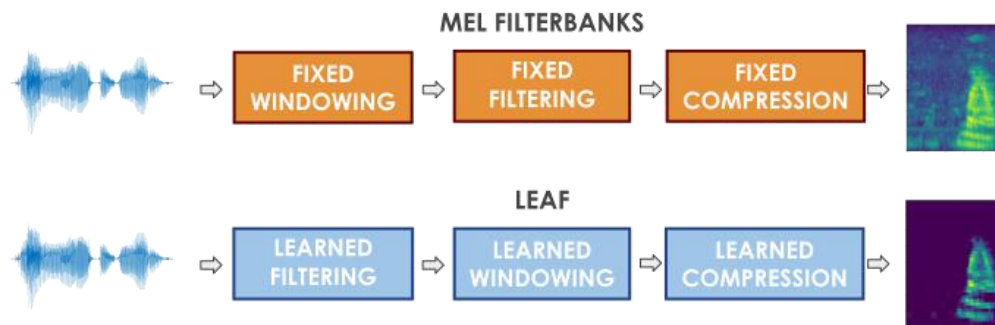
Part I: how to encode audio with Neural Networks?

- Option 1: raw audio signal
 - Use 1-dimensional convolutions
 - Learn your own filters along with your NN
 - More tricky than using log-Mel coefficients directly, though, and not always better
 - Methods: SincNet (2018), wav2vec (2018), LEAF (LEarnable Audio Frontend, March 2021...)

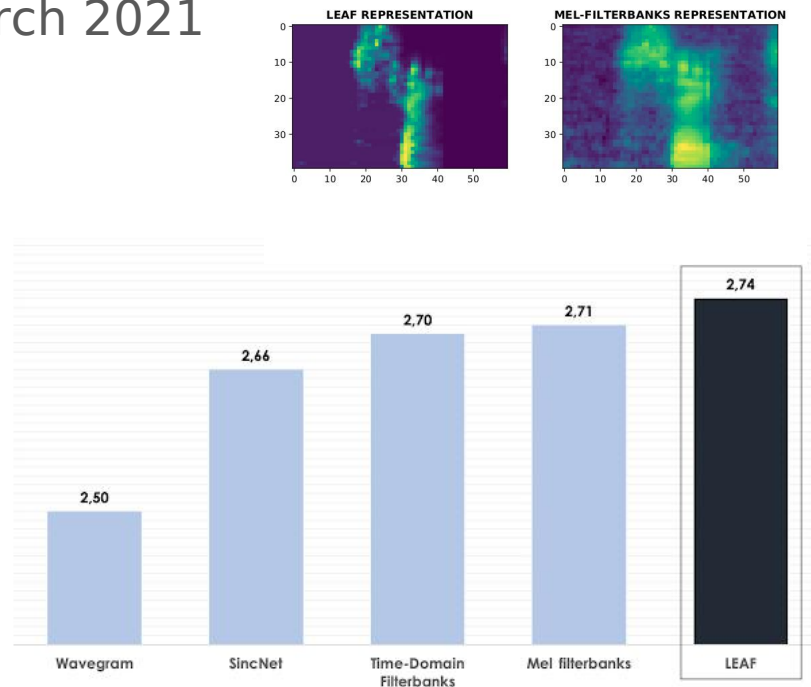
How to encode audio with Neural Networks?

- Option 1: raw audio signal

→LEAF, LEarnable Audio Frontend, March 2021

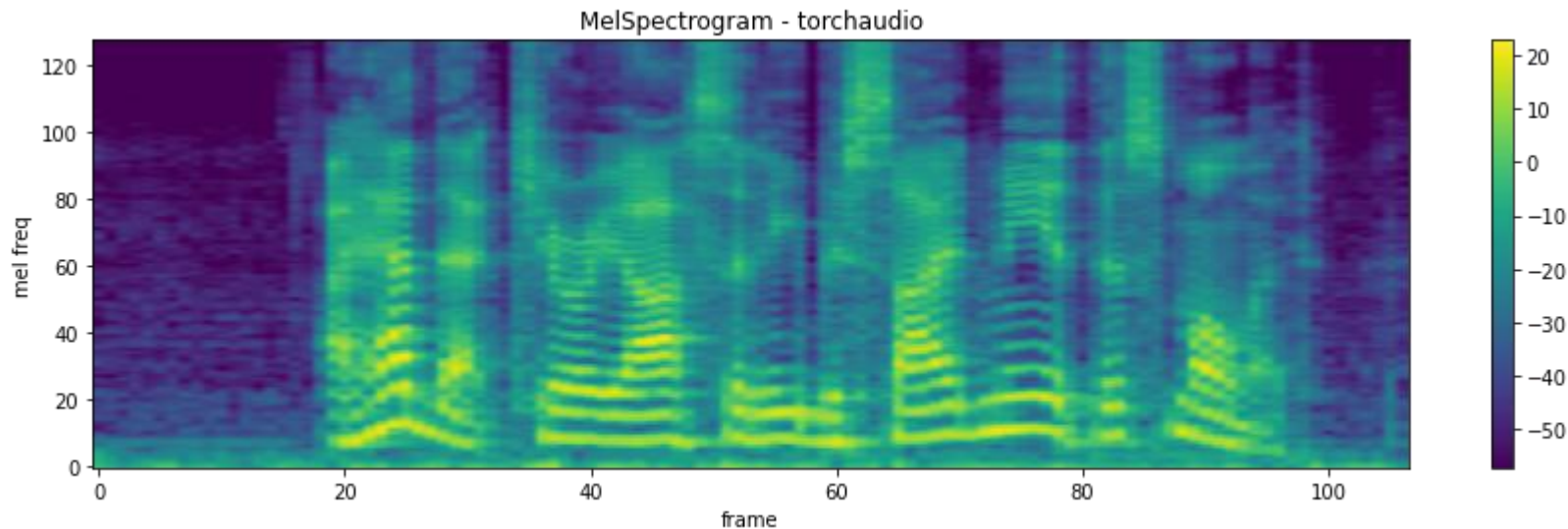


AudioSet, d-prime
score



How to encode audio with Neural Networks?

- Option 2: log Mel coefficients → audio data processed as **images**



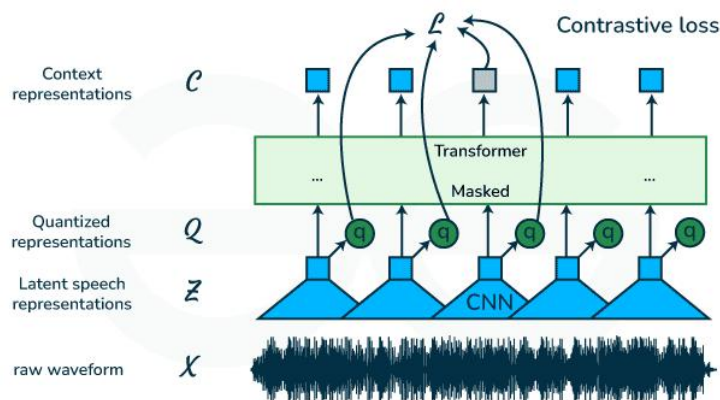
How to encode audio with Neural Networks?

Wav2vec, Facebook AI Research, 2018

Self-supervised learning model for speech representation learning

- learn directly from raw waveforms
- predicting future audio segments given past information
- first pre-train on unlabelled data, then finetune on labelled data

like ChatGPT!



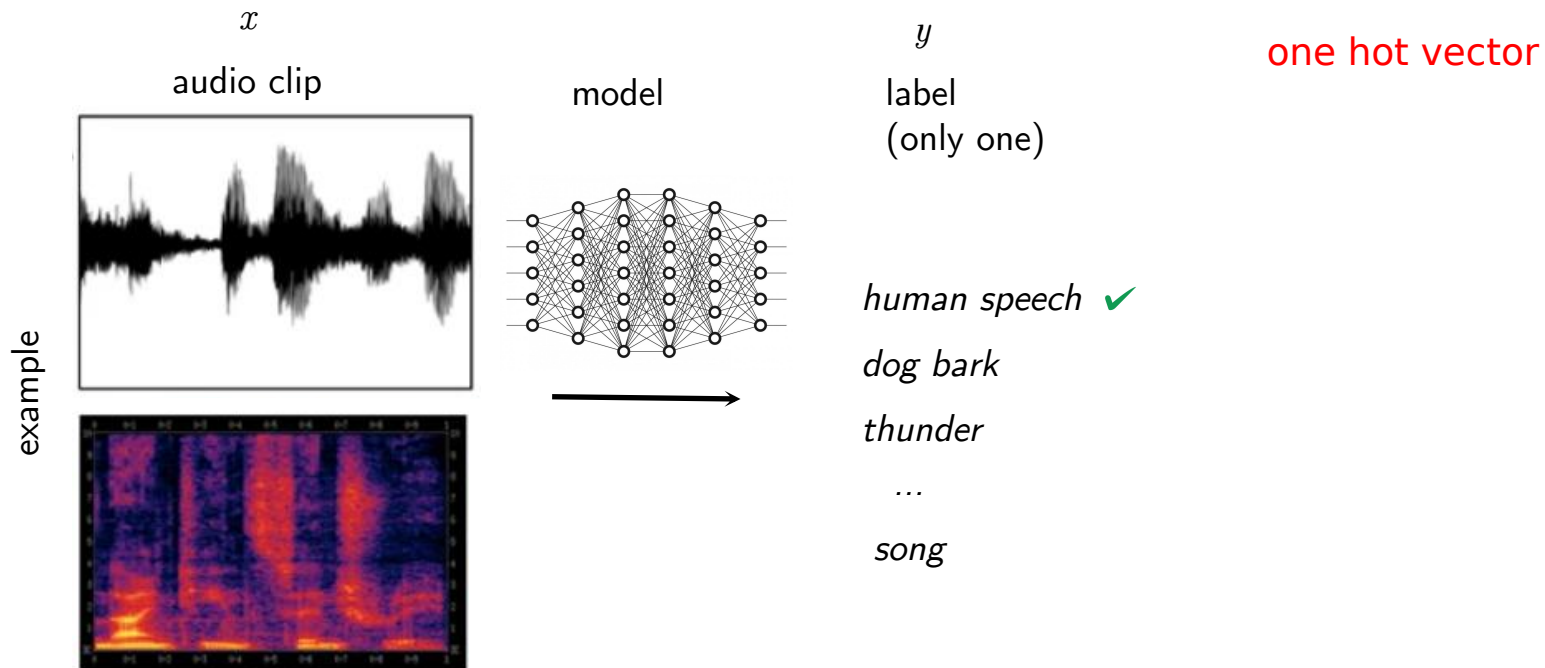
IV. Processing audio

with artificial neural networks

The next following slides contain figures/material from
<https://www.cse.iitb.ac.in/~pjyothi/cs753/index.html>

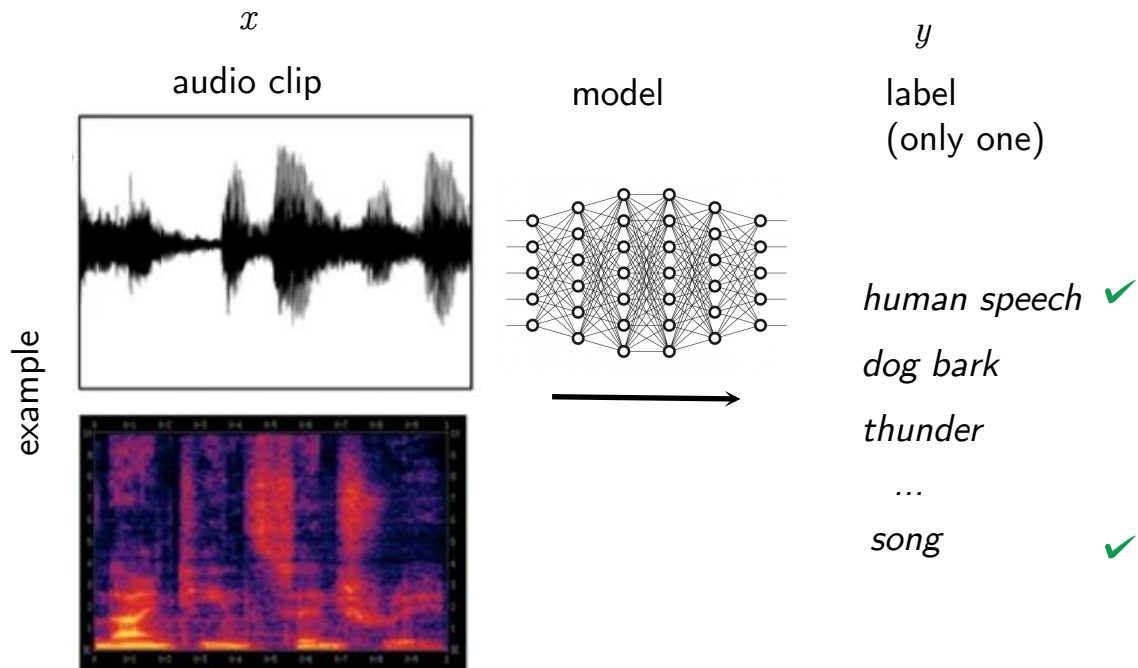
Classical audio processing tasks

- audio **classification**: assign one class to an input clip



Classical audio processing tasks

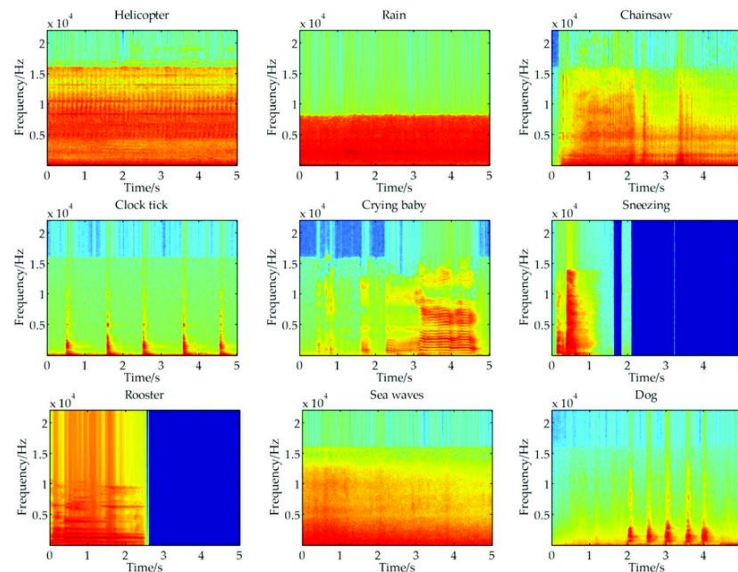
- audio **tagging**: assign multiple class to an input clip = “multi-class classification”



AudioSet dataset

THE big audio dataset

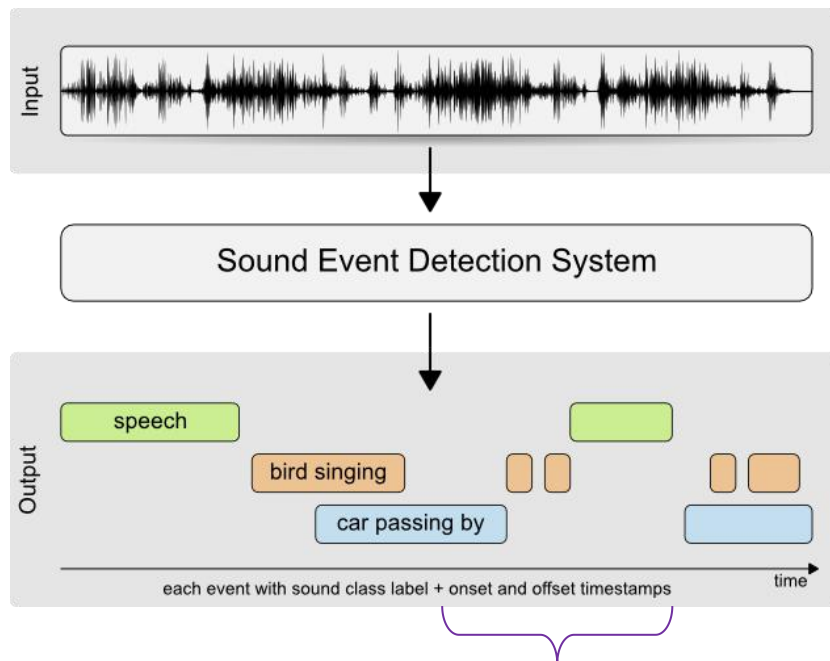
- equivalent of **ImageNet for audio**
- 2 To of sound clips scrapped on YouTube
- **multilabel classification**
- hundreds of classes (speech, music, environmental sounds, ...)



<https://research.google.com/audioset/>

Classical audio processing tasks

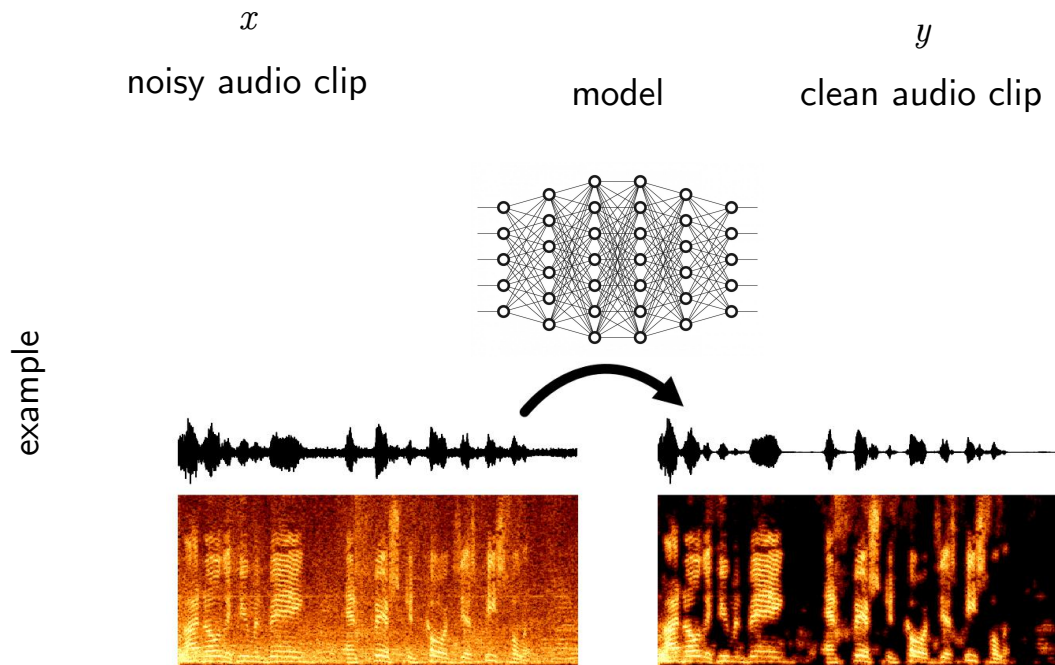
- **event detection:** a bit like audio tagging, but on a continuous stream



each event is considered a **single** object (cf. instance segmentation in computer vision)

Classical audio processing tasks

- **denoising**

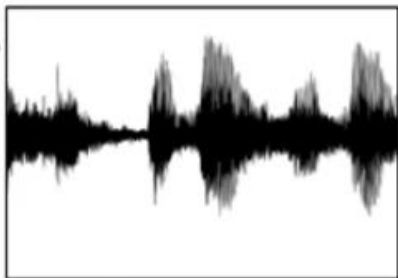


Classical audio processing tasks

- **Automatic speech recognition (ASR):** convert audio to text (probably the most difficult of these tasks)

x

audio clip

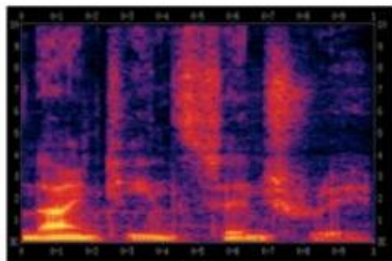


y

transcript (text)

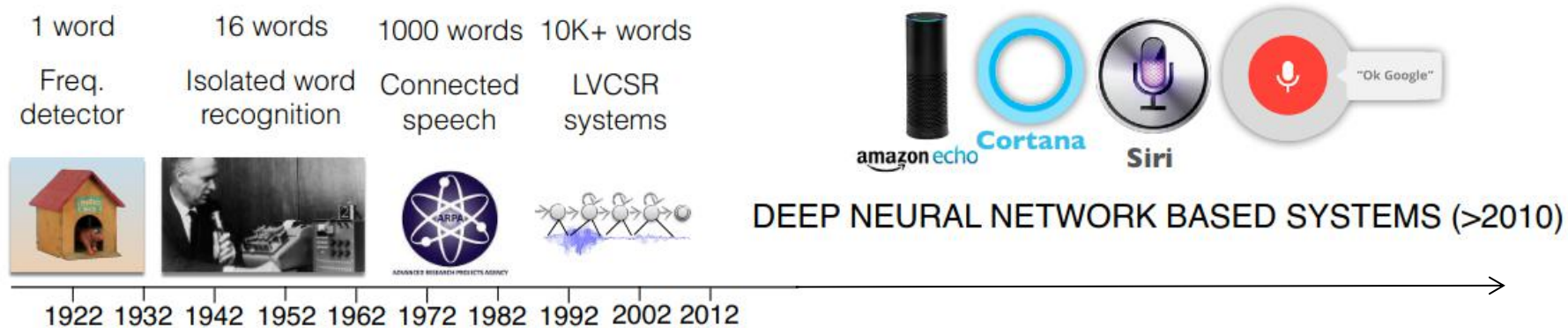
"the quick brown fox"

example

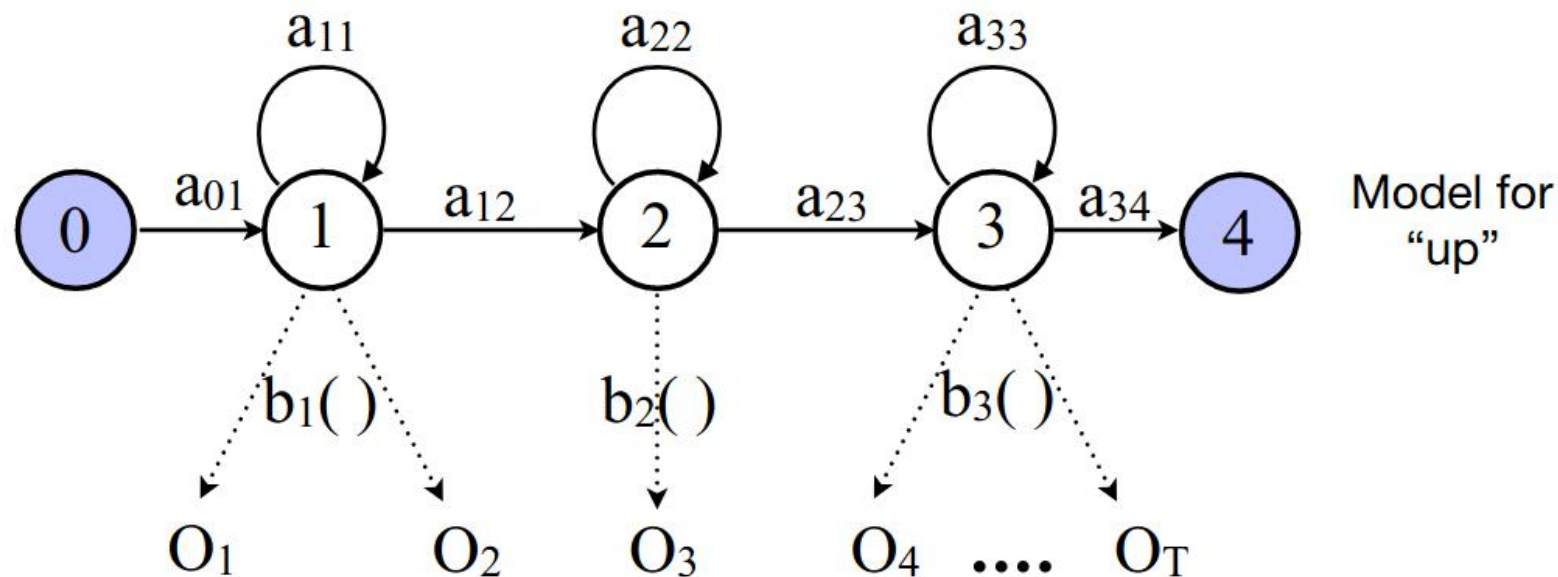


inspired by DeepLearning.AI 's course **C5W3L09 SpeechRecog** (by Andrew Ng)
<https://www.youtube.com/watch?v=vm2Si8AJY0s>

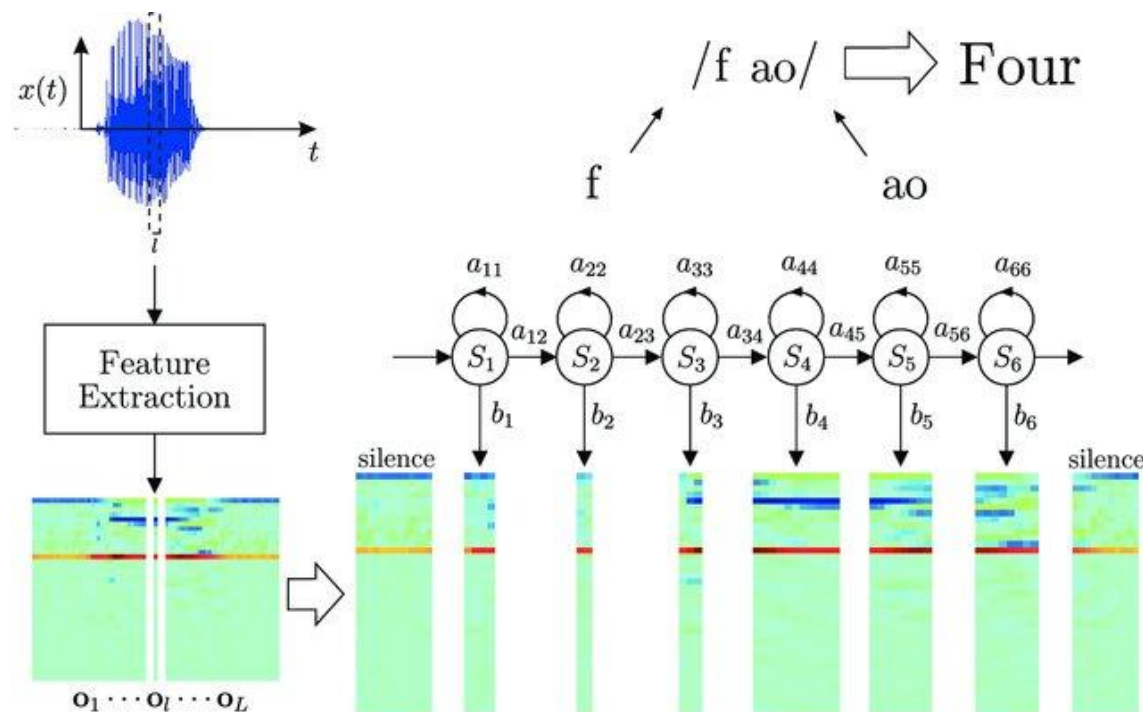
ASR: historic overview



Isolated word recognition

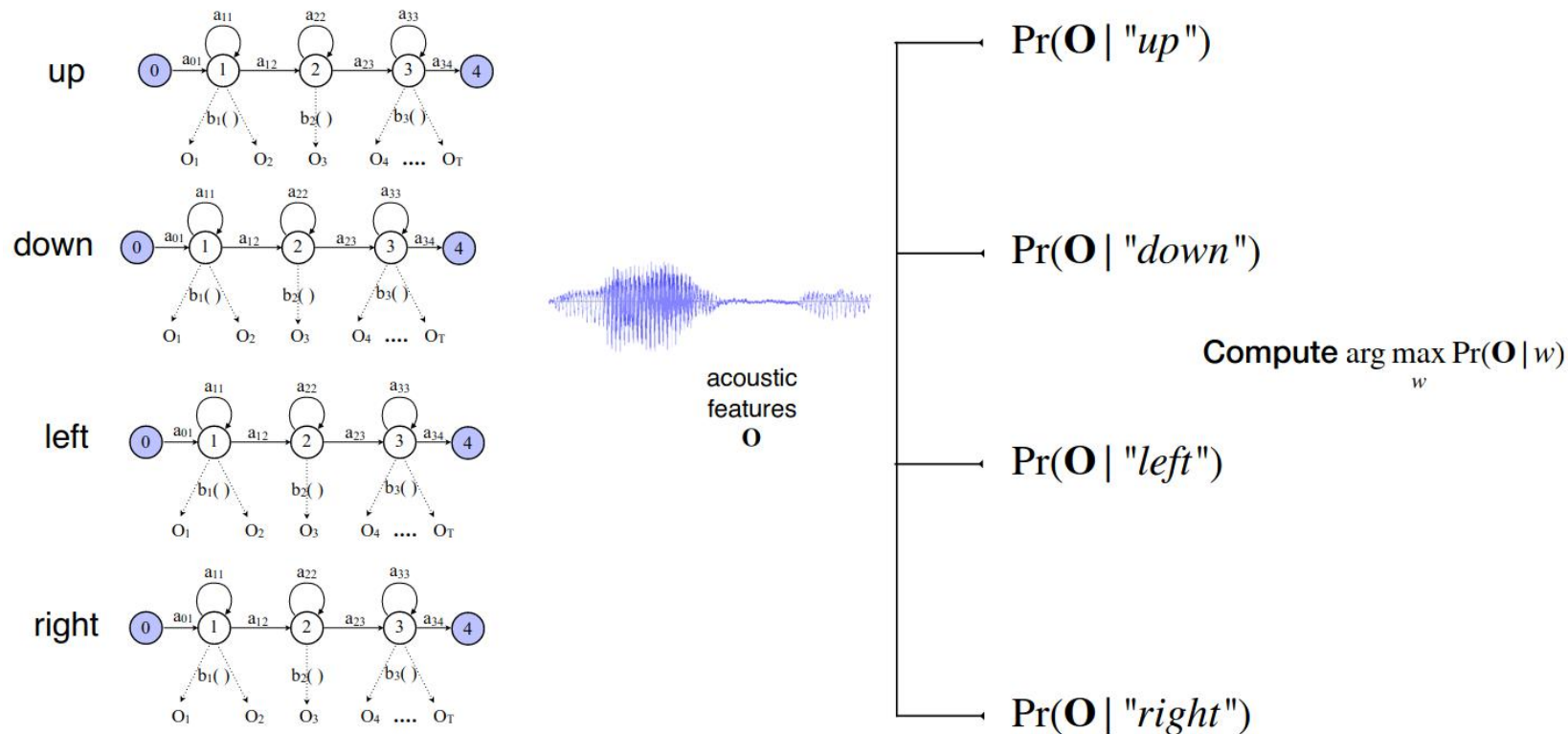


Isolated word recognition

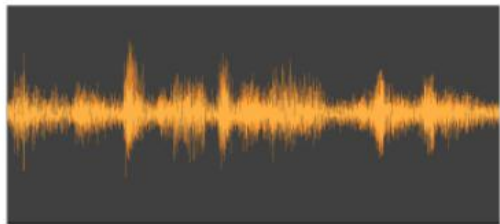


Fernández Astudillo, R. (2010). Integration of Short-Time Fourier Domain Speech Enhancement and Observation Uncertainty Techniques for Robust Automatic Speech Recognition.

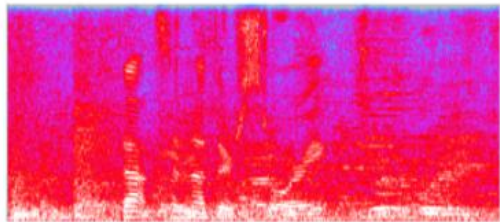
Isolated word recognition



ASR in 1970s-: a noisy channel model



$$X = \{x_l \in \mathbb{Z} | l = 1, \dots, L\}$$
$$L = 43263$$



$$X = \{\mathbf{x}_t \in \mathbb{C}^D | t = 1, \dots, T\}$$
$$T = 268$$



“That’s another story”

$$W = \{w_n \in \mathcal{V} | n = 1, \dots, N\}$$
$$N = 3$$

https://deeplearning.cs.cmu.edu/F20/document/slides/shinji_watanabe_e2e_asr_bhiksha.pdf

ASR in 1970s-: a noisy channel model

$$\arg \max_W p(W|X)$$

X : Speech sequence

W : Text sequence

https://deeplearning.cs.cmu.edu/F20/document/slides/shinji_watanabe_e2e_asr_bhiksha.pdf

ASR in 1970s-: a noisy channel model

L : Phoneme sequence

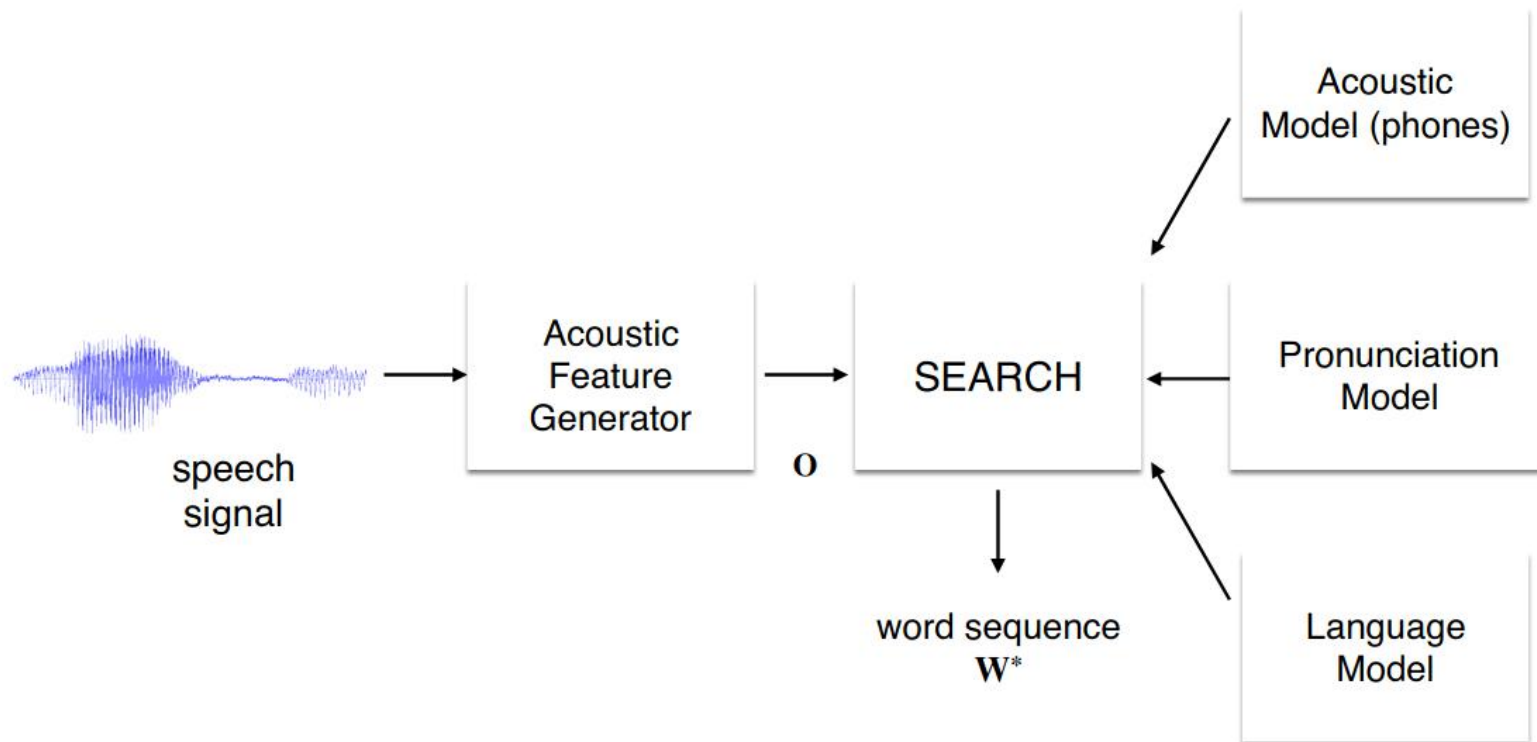
$$\begin{aligned}\arg \max_W p(W|X) &= \arg \max_W p(X|W)p(W) \\ &\approx \arg \max_{W,L} p(X|L, W)p(L|W)p(W)\end{aligned}$$

- **Speech recognition**

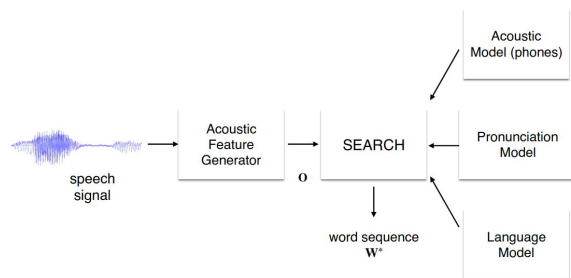
- $p(X|L)$: Acoustic model (Hidden Markov model)
- $p(L|W)$: Lexicon
- $p(W)$: Language model (n-gram)

https://deeplearning.cs.cmu.edu/F20/document/slides/shinji_watanabe_e2e_asr_bhiksha.pdf

Standard ASR systems from 1980 to ~2010



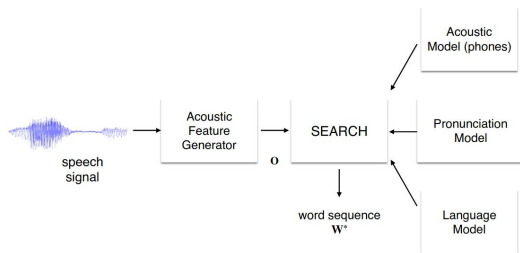
Standard ASR systems from 1980 to ~2010



A lot of research and engineering in there:

- Acoustic models of phones: tied-state Hidden Markov Models with continuous observation density functions (Gaussian Mixture Models)
- Phonetic decisions trees to model context-dependent phones
- Language models: n-grams with backoff strategies
- Composed Weighted Finite Transducers to integrate the AMs, PM and LM into a single finite state machine
- Very clever algorithms: Viterbi, Baum-Welch, Expectation Maximization, Beam search algorithms...
- Expert linguistic knowledge to build a pronunciation lexicon

Standard ASR systems from 1980 to ~2010

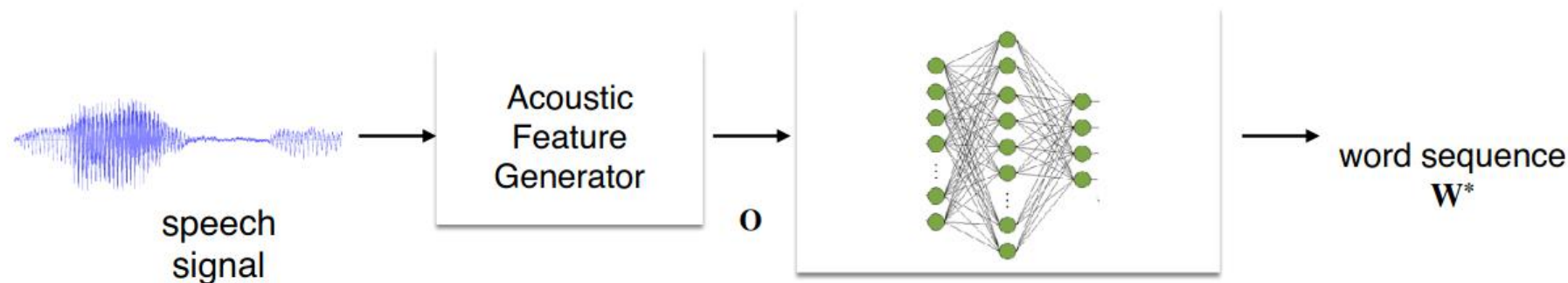


A lot of research and engineering in there:

- Acoustic models of phones: tied-state Hidden Markov Models with continuous observation density functions (Gaussian Mixture Models)
- Phonetic decisions trees to model context-dependent phones
- Language models: n-grams with backoff strategies
- Composed Weighted Finite Transducers to integrate the AMs, PM and LM into a single finite state machine
- Very clever algorithms: Viterbi, Baum-Welch, Expectation Maximization, Beam search algorithms...
- Expert linguistic knowledge to build a pronunciation lexicon

**Big barrier for ASR
non-experts!**

End-to-end ASR systems >2010



Single end-to-end model that directly learns a mapping from speech to text

ASR: a solved problem?

Voice Recognition Software Finally Beats Humans At Typing, Study Finds

AUG '16



Microsoft researchers achieve new conversational speech recognition milestone

AUG '17



Amazon's AI system could cut Alexa speech recognition errors by 15%

MAR '19



<https://venturebeat.com/2019/04/22/amazons-ai-system-could-cut-alexa-speech-recognition-errors-by-15/>
<https://www.microsoft.com/en-us/research/blog/microsoft-researchers-achieve-new-conversational-speech-recognition-milestone/>
<https://www.npr.org/sections/alltechconsidered/2016/08/24/491156218/voice-recognition-software-finally-beats-humans-at-typing-study-finds>

What are some unsolved problems related to ASR?

- State-of-the-art ASR systems do not work well on regional accents, dialects
- Code-switching is hard for ASR systems to deal with
- How do we rapidly build competitive ASR systems for a new language? Low-resource ASR and keyword spotting
- How do we recognize speech from meetings where a primary speaker is speaking amidst other speakers? Babble noise, overlapped speech

CNNs for speech processing

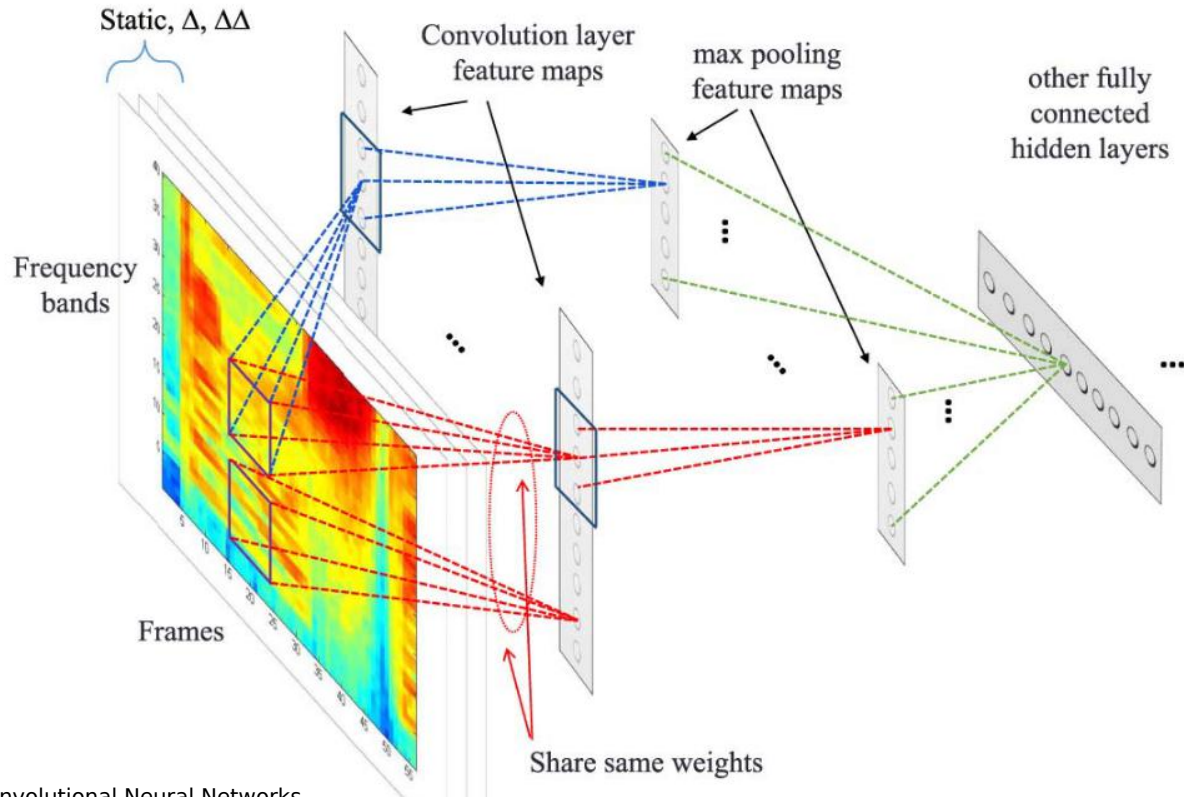
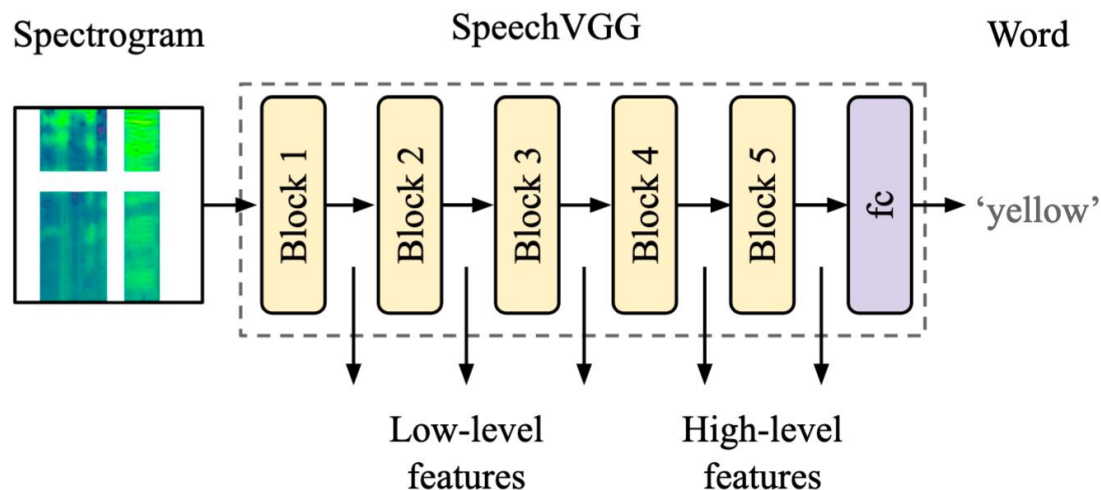


Image from Abdel-Hamid et al., "Convolutional Neural Networks for Speech Recognition", TASLP 2014

Speech-VGG



- Trained to predict the 1000 most frequent words from LibriSpeech
- Then used as a feature extractor for downstream tasks

Beckmann, Pierre, et al. "Speech-vgg: A deep feature extractor for speech processing." *arXiv preprint arXiv:1910.09909* (2019).

Speech-VGG

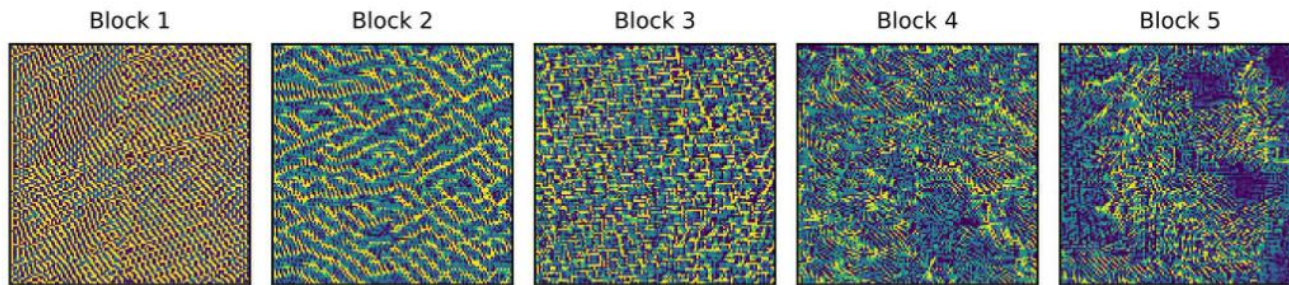
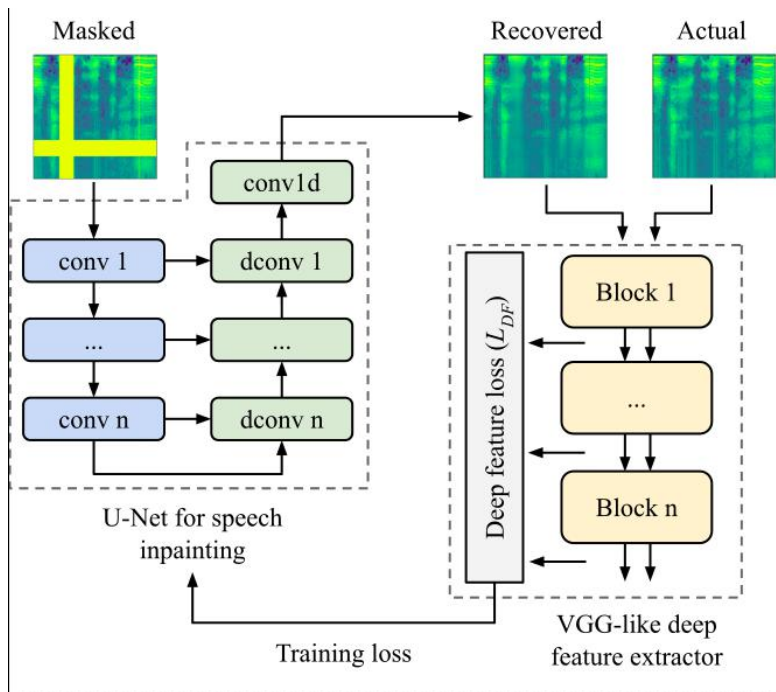


Figure 3: *Sample inputs that maximize activation of pooling layers, at the end of each block, of the pre-trained speechVGG.*

Beckmann, Pierre, et al. "Speech-vgg: A deep feature extractor for speech processing." *arXiv preprint arXiv:1910.09909* (2019).

Application example: speech inpainting



Demo:

<https://mkegler.github.io/SpeechInpainting/>

Beckmann, Pierre, et al. "Speech-vgg: A deep feature extractor for speech processing." *arXiv preprint arXiv:1910.09909* (2019).

Transfer Learning

Use knowledge learned on a task for another one.

1. **pre-train** a model to classify sounds on AudioSet dataset --> general knowledge of all sounds
2. **fine-tune** on a more specific, downstream task (e.g. ASR) --> “expert” model

Speech is sequential

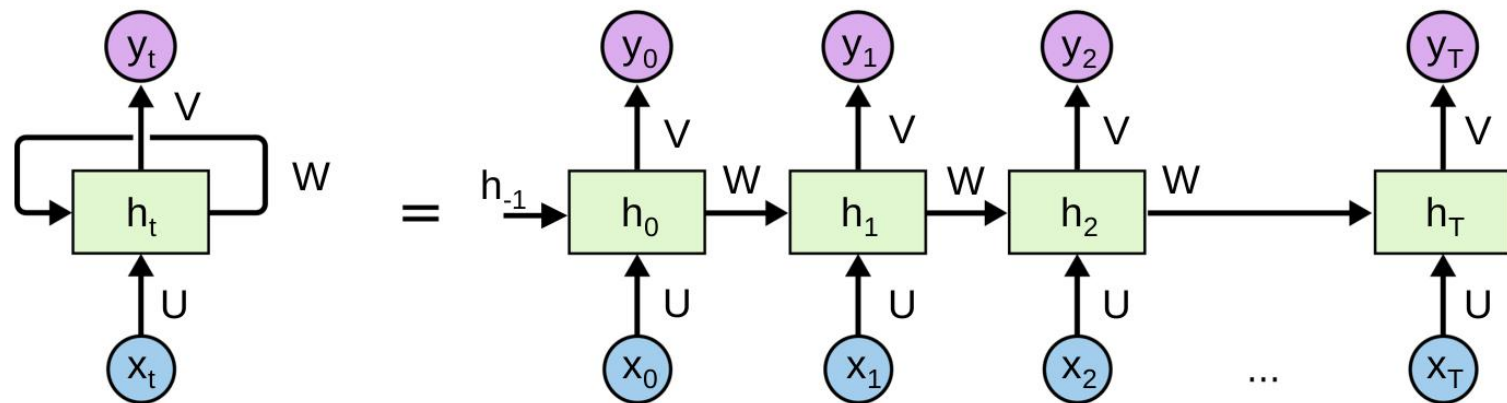
CNNs are OK for short speech excerpts. But how to handle longer speech recordings ?

→ Use Recurrent Neural Networks (RNNs)

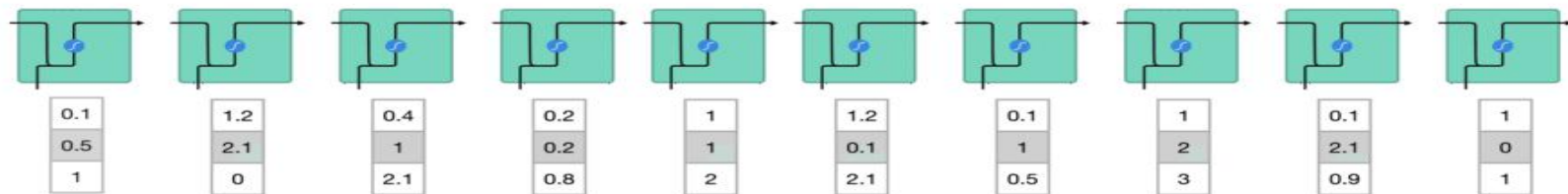
Speech is sequential

CNNs are OK for short speech excerpts

→ Use Recurrent Neural Networks instead



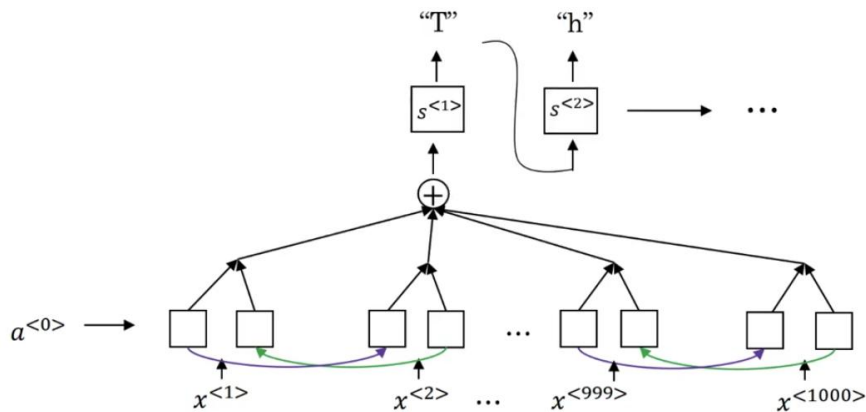
Speech is sequential (RNNs)



Credit : Animated RNN, LSTM and GRU, Michaël Nguyen.

Connectionist Temporal Classification (CTC)

An efficient loss for ASR, introduced by Graves et al. (2006)

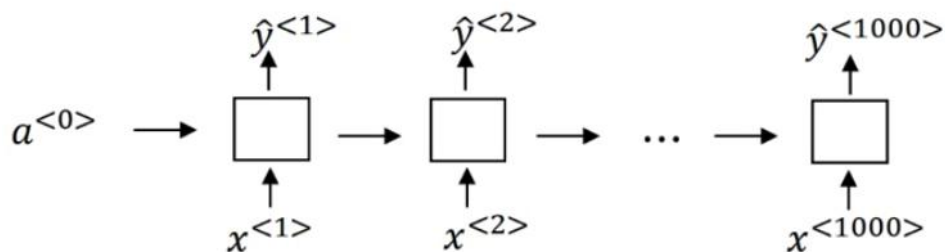


Problem: much more inputs than outputs !!

inspired by DeepLearning.AI 's course **C5W3L09 SpeechRecog** (by Andrew Ng)
<https://www.youtube.com/watch?v=vm2SI8AIY0s>

CTC objective

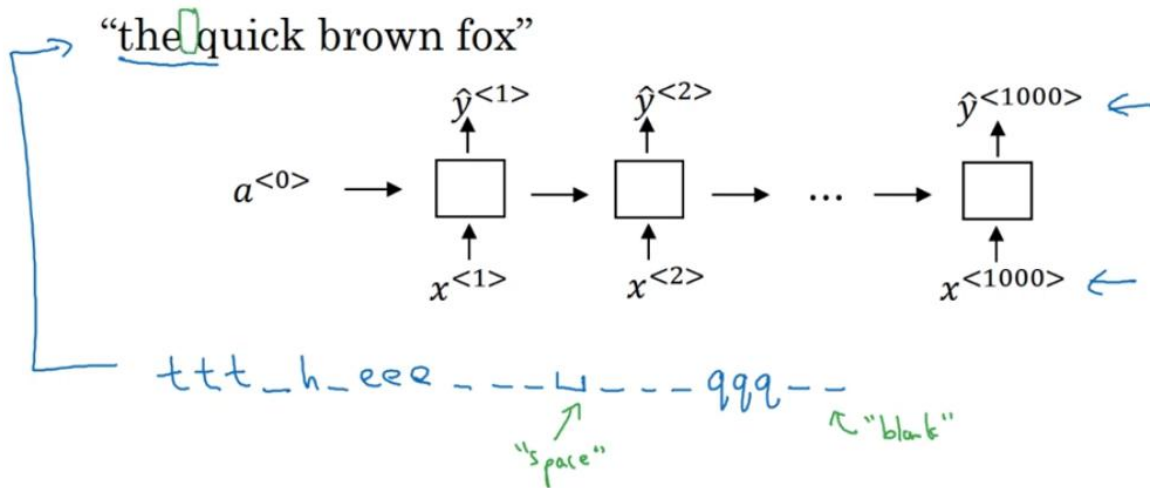
“the quick brown fox”



Problem: much more inputs than outputs !!

inspired by DeepLearning.AI 's course **C5W3L09 SpeechRecog** (by Andrew Ng)
<https://www.youtube.com/watch?v=vm2Sl8AJY0s>

CTC objective



Basic rule: collapse repeated characters not separated by “blank”

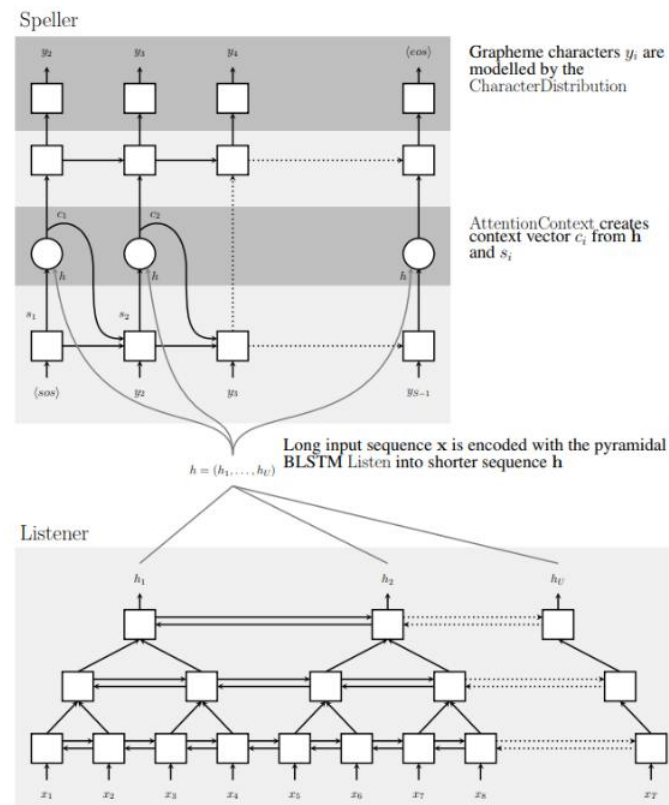
inspired by DeepLearning.AI 's course **C5W3L09 SpeechRecog** (by Andrew Ng)
<https://www.youtube.com/watch?v=vm2SI8AJY0s>

Speech is sequential

Listen-Attend-Spell (LAS) Chan et al. 2015

Encoder-Decoder
with attention

- ▶ Listen:
 $\mathbf{h} = \text{BLSTM}(\mathbf{x})$
- ▶ Attend:
 $c_i =$
 $\text{AttentionContext}(s_i, \mathbf{h})$
- ▶ Spell:
 $s_i =$
 $\text{RNN}(s_{i-1}, y_{i-1}, c_{i-1})$



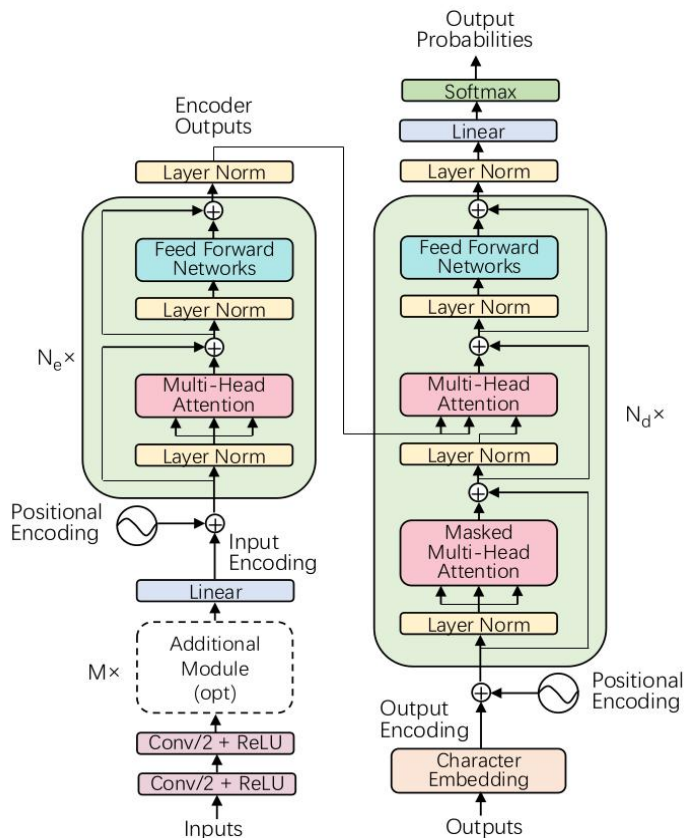
ASR and Transformers

Vaswani et al. **2017**

Transformers

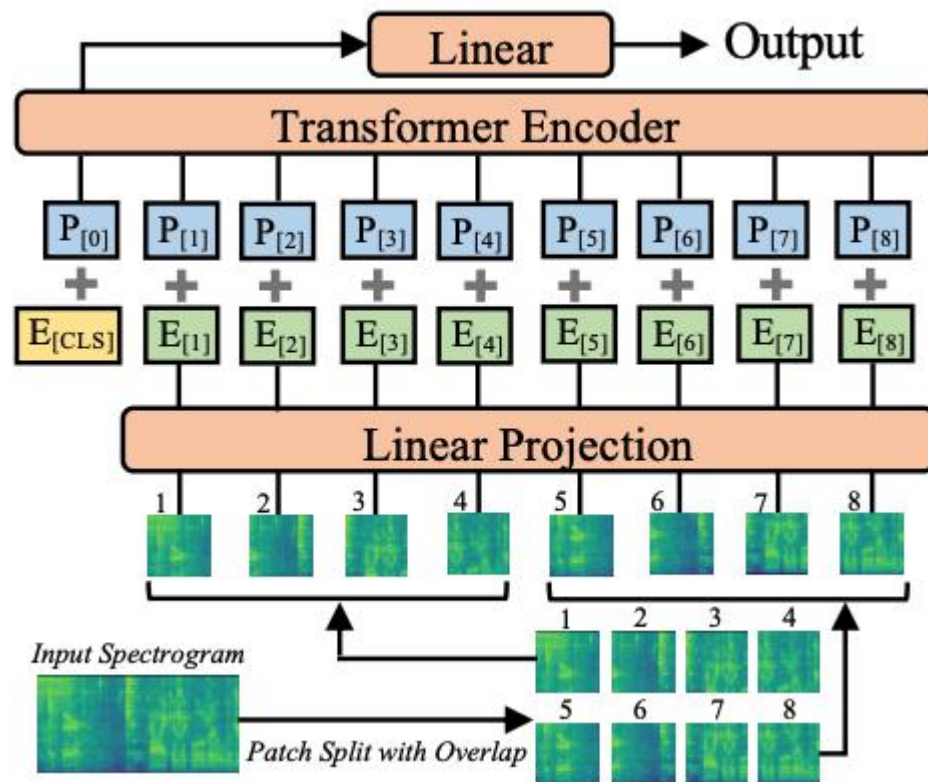
Dong et al. **2018**

Speech transformer



$$z_j = \sum_i \text{Softmax} \left(\frac{QK}{\sqrt{d}} \right)_i v_{i,j}$$

ASR and Transformers



$$z_j = \sum_i \text{Softmax} \left(\frac{QK}{\sqrt{d}} \right)_i V_{i,j}$$

ASR: some of the latest SOTA

Transformers take over major AI applications

- Giant *Conformers* pre-trained using wav2vec 2.0 and self-training achieve the lowest word-error rates (WER) to date on Librispeech.
- The *Conformer* model combines **self-attention** and **convolutions** to capture both global interactions and local features.

Google researchers tackle the high-resource language degradation problem by **increasing model capacity**.

- The underlying assumption in Multilingual ASR (Automatic Speech Recognition) is that the additional information learned **from one language** should benefit **other languages**. In practice, using more languages makes the modeling task more difficult due to **large language variations** and heavy data imbalance.
- While low-resource languages do benefit from multilingual training, high resource languages (like English) usually suffer from the reduction in model capacity compared to the monolingual setting.

Nathan Benaich and Ian Hogarth, State of AI Report
October 12, 2021

Beyond speech recognition...

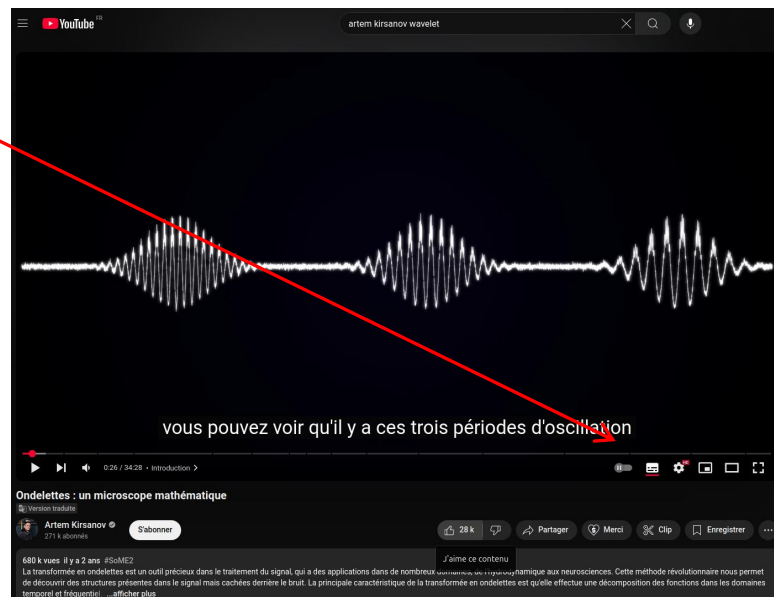
Speech recognition is more or less solved...
(cf. youtube's automatic transcription)

But other audio tasks remain!

- speech enhancement / **denoising**
- speech **synthesis** (vocoders)
- text to audio (audio counterpart of big image models)

e.g., <https://ttsmaker.com/>

Mostly **generative AI** (> 2020s)



V. Hands-on practice

code your own automatic audio recognition model !

Task description

TorchAudio's Automatic speech recognition (ASR) tutorial on Google Speech Commands

https://colab.research.google.com/drive/1z51RHWyMED0eKjQzA1VLgzvdPN2fGxp5?usp=sharing#scrollTo=_vfJAE7xPUON



Grading

Grading will be based on **individual projects**. Each student should choose **ONE** of the course notebooks (from lectures 1-9) and complement it with a *significant* personal work. Just create one or more cells at the end of the notebook, clearly marked with your personal addition. It could be a change in the model's architecture, an adaptation of the model to a new task or dataset, etc. If you lack inspiration, feel free to start from the "exercises" or personal work suggested at the end of each notebook (but projects with original and creative thinking will get extra points). The link to the colab notebook (**after** making sure that it is "*shared for anyone with the link*") should be submitted by email to rufin.vanrullen@cnrs.fr on or before November 15. This is a firm deadline, the grade will decrease by 1 point for each late day (e.g. if you send your notebook on November 25, the maximum grade you can get is 10/20). The main criteria for grading will be:

- The addition should be clearly marked in the notebook, e.g. with a title "MY PERSONAL ADDITION IN THE 3 CELLS BELOW"
- The addition should be explained (e.g. with comments in the code), so one can at least know what you intended to achieve
- The addition should be non-trivial
- The addition should reflect significant work and effort
- The addition should make it clear that the student has understood the lecture and notebook
- The addition should *work*, i.e. it should be possible to evaluate the cells without returning an error.

Exercise

```
class ConvModel(nn.Module):
    def __init__(self, n_input=1, n_output=35, n_channel=64):
        super().__init__()

        self.conv1 = nn.Conv2d(n_input, n_channel, kernel_size = 3, padding = 1) # X * W + b
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(n_channel, n_channel, kernel_size = 3, padding = 1)
        self.conv3 = nn.Conv2d(n_channel, n_channel, kernel_size = 3, padding = 1)
        self.fc1 = nn.Linear(n_channel * (40 // 2**3) * (80 // 2**3), 120)
        self.fc2 = nn.Linear(120, n_output)

    # TO DO:
    def forward(self, x):
        # Batch size (128) , chan_in (1), 40, 80
        # apply the first layer : conv + ReLU + max pool
        x = self.conv1(x)
        x = F.relu(x)
        x = self.pool(x)

        # apply the second layer : conv + ReLU + max pool
        x = self.conv2(x)
        x = F.relu(x)
        x = self.pool(x)

        # apply the third layer : conv + ReLU + max pool
        x = self.conv3(x)
        x = F.relu(x)
        x = self.pool(x)

        # size of x ? = # Batch size (128) , ch (64), 5, 10
        # flatten all dimensions except batch 128,64,5,10 -> 128,(64*5*10)
        x = torch.flatten(x, start_dim=1)

        # apply the first FC module + ReLU
        x = self.fc1(x)
        x = F.relu(x)

        # apply the last FC module
        x = self.fc2(x)

        return x
```

Based on the code, draw a schematic of the model architecture:

Programming assignment ideas

Change the model architecture:

- Audio front-end (gammatone filters, MFCCs)
- CNN --> RNN (LSTM/GRU...)
- “home-made” --> off-the-shelf model

Change the training itself:

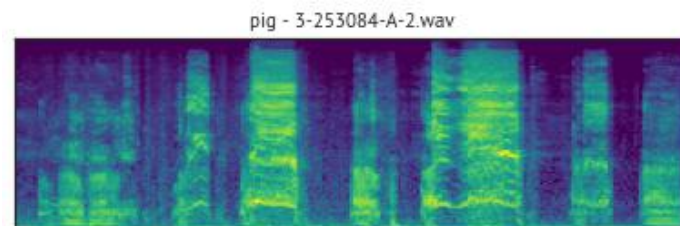
- data augmentation
- dropout
- learning rate scheduler

Change dataset ! A few examples:

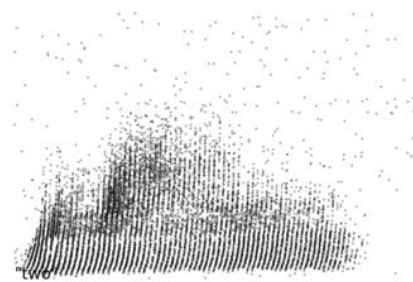
- ESC-50
- Spiking Heidelberg Digits (SHD)

Analyze your results “like a researcher” and present some nice plots:

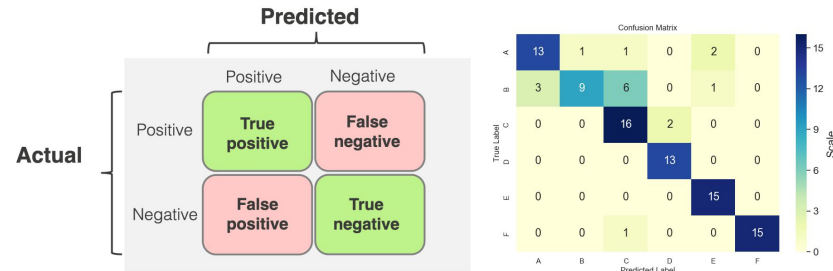
- confusion matrix
- compute model’s receptive fields



ESC50 sample



SHD sample



Going further...

A very nice course on audio processing with Transformers:

<https://huggingface.co/learn/audio-course/>



HUGGING FACE

Conclusion

Thank you for your attention !

Future improvements

Future additions / clarifications to this presentation:

- (binaural) sound source localization + biological inspiration (neuroscience)
- common data augmentations in audio (eg, time masking, frequency masking...)